TITRE PROFESSIONNEL

DÉVELOPPEUR WEB ET WEB MOBILE (DWWM)

DOSSIER DE PROJET

ITConnect

Plateforme de mise en relation entre particuliers, entreprises et professionnels de l'informatique

Conception, développement, tests, sécurité, RGPD et déploiement

Candidat : **Loïc Léotet**

Session : **2026**

Centre de formation : Centre Européen de Formation

Octobre 2026

Sommaire

La pagination débute à la Partie 1.

Partie 1 — Présentation du projet et de son contexte 1

1.1 Contexte et origine du projet 1
1.2 Objectifs du projet 1
1.3 Utilisateurs et rôles 1
1.4 Périmètre fonctionnel 2
1.5 Contraintes 2
1.6 Livrables attendus 2

Partie 2 — Analyse et conception 3

2.1 Recueil et analyse du besoin 3
2.2 Spécifications fonctionnelles 3
2.3 Matrice des droits par rôle 3
2.4 Cas d'utilisation 4
2.5 Architecture technique 4
2.6 Justification des choix technologiques 5
2.7 Maquettage et charte graphique 5

Partie 3 — Conception et réalisation de la base de données 9

3.1 Démarche de modélisation 9
3.2 Modèle conceptuel de données (MCD) 9
3.3 Diagramme de classes 9
3.4 Modèle logique relationnel 10
3.5 Dictionnaire de données 11
3.6 Relations et règles de gestion 12
3.7 Création, configuration et données de démonstration 12

Partie 4 — Réalisation technique 14

4.1 Environnement et outils de développement 14
4.2 Organisation du code 14
4.3 Réalisation du back-end 14
4.4 Réalisation du front-end 17
4.5 Fonctionnement global : lien entre le front-end et le back-end 18
4.6 Interfaces réalisées 20
4.7 Interface responsive 22
4.8 Accessibilité et référencement 24

Partie 5 — Tests et recette 25

5.1 Stratégie de test 25
5.2 Tests unitaires 25
5.3 Tests fonctionnels de l'API avec Postman 26
5.4 Recette de l'interface 27
5.5 Compatibilité navigateurs et responsive 27
5.6 Bilan des tests et axes d'amélioration 27

Partie 6 — Sécurité : failles majeures et protections 29

6.1 Cartographie des menaces 29
6.2 Injection SQL 29
6.3 Cross-Site Scripting (XSS) 29
6.4 Cross-Site Request Forgery (CSRF) 30
6.5 Authentification et gestion des sessions 30
6.6 Contrôle d'accès et « mass assignment » 30
6.7 Exposition de données et configuration 30
6.8 Limites et améliorations prévues 31

Partie 7 — Conformité au RGPD 32
7.1 Cadre réglementaire 32
7.2 Données collectées, finalités et conservation 32
7.3 Principes appliqués 32
7.4 Droits des personnes et mise en œuvre technique 33
7.5 Sécurité des données et gestion d'une violation 33
7.6 Cookies et traceurs 33
7.7 Sous-traitants et transferts 33
7.8 Actions restantes avant une exploitation réelle 33

Partie 8 — Déploiement et hébergement 35
8.1 Vue d'ensemble des environnements 35
8.2 Environnement local 35
8.3 Publication du front-end sur GitHub Pages 36
8.4 Déploiement de l'API sur Render 37
8.5 Variables d'environnement par environnement 37
8.6 Versionnement et flux de branches 38
8.7 Liste de contrôle avant mise en production 38
8.8 Maintenance 38

Partie 9 — Gestion de projet 39
9.1 Méthodologie 39
9.2 Planning 39
9.3 Outils de suivi 39
9.4 Veille technologique 39
9.5 Budget indicatif de mise en production 39

Partie 10 — Difficultés rencontrées et solutions apportées 40
10.1 Analyse transversale 40

Partie 11 — Bilan, prise de recul et conclusion générale 41
11.1 Bilan technique : objectifs atteints 41
11.2 Compétences du titre mobilisées 41
11.3 Prise de recul 41
11.4 Perspectives d'évolution 42
11.5 Conclusion générale 42

Annexes 43
Annexe A — Index des routes de l'API 43
Annexe B — Variables d'environnement (extrait du README) 43
Annexe C — Glossaire 43

Partie 1 — Présentation du projet et de son contexte

1.1 Contexte et origine du projet

ITConnect est une plateforme web de mise en relation entre des **particuliers**, des **entreprises** et des **professionnels indépendants de l'informatique** (dépannage, développement, administration systèmes et réseaux, cybersécurité, data, UX design). Elle répond à un double constat : les particuliers et les petites structures peinent à trouver rapidement un professionnel informatique fiable, disponible et proche pour un besoin ponctuel ; à l'inverse, les freelances et petites structures IT manquent de visibilité auprès d'une clientèle locale sans dépendre de plateformes généralistes peu adaptées à leur métier.

Le projet a été réalisé **seul**, dans le cadre de la préparation du titre professionnel **Développeur Web et Web Mobile (DWWM)**. ITConnect est une entreprise fictive (SAS basée à Auxerre) servant de cadre professionnel réaliste : elle fournit des utilisateurs, des contraintes et des objectifs qui guident les choix de conception. La plateforme affiche les zones couvertes (Melun, Sens, Auxerre, Montereau, Montpellier, Bordeaux) et centralise recherche, demandes de prestation et avis.

1.2 Objectifs du projet

- Permettre à un particulier ou à une entreprise de **rechercher** un professionnel IT par spécialité, ville, disponibilité et note.
- Permettre à un professionnel de **créer et gérer son profil** (compétences, tarif, disponibilité) et de recevoir des demandes.
- Fournir un **suivi des demandes** de leur dépôt à leur clôture (statuts En attente, En cours, Terminée, Refusée).
- Instaurer la **confiance** grâce à un système d'avis et de notation (un avis par client et par professionnel).
- **Sécuriser** les accès et les données : authentification par jeton, contrôle d'accès par rôle, protection contre les failles majeures.
- Proposer une interface **moderne, responsive et accessible** sur ordinateur, tablette et mobile.

1.3 Utilisateurs et rôles

La plateforme distingue quatre rôles, stockés dans le champ `role` de la table des utilisateurs. Les trois premiers peuvent être choisis à l'inscription ; le rôle administrateur n'est jamais attribuable par l'API (voir 6.6).

Rôle	Profil type	Besoin principal	Actions principales
Particulier	Camille, 35 ans, peu à l'aise avec l'informatique	Trouver un dépanneur de confiance	Rechercher, consulter un profil, déposer une demande, laisser un avis
Entreprise	TPE de 6 salariés	Externaliser la maintenance informatique	Rechercher, déposer des demandes, suivre ses prestataires, laisser un avis
Professionnel IT	Nadia, développeuse freelance	Développer sa clientèle et sa visibilité	Gérer son profil, consulter et traiter les demandes reçues, suivre ses avis
Administrateur	Équipe ITConnect	Garantir la qualité de la plateforme	Compte créé via le script de la base ; modération prévue dans une version ultérieure

1.4 Périmètre fonctionnel

La version réalisée couvre le **parcours complet** d'un utilisateur, de l'inscription à la clôture d'une demande : création de compte et connexion, recherche avec filtres, consultation de profils, dépôt et suivi de demandes, dépôt d'avis, tableau de bord adapté au rôle et suppression de compte. Sont **hors périmètre** de cette version : le paiement en ligne, la prise de rendez-vous intégrée, l'application mobile native et l'interface de modération administrateur (présentés en perspectives, partie 11).

1.5 Contraintes

Nature	Contrainte	Conséquence sur le projet
Technique	Technologies open source uniquement (React, Node.js, MySQL)	Maîtrise des coûts, large documentation
Technique	Application utilisable sur ordinateur, tablette et smartphone sans développement natif	Interface responsive, tests sur trois largeurs d'écran
Réglementaire	Protection des données personnelles (RGPD)	Minimisation, hachage des mots de passe, droit à l'effacement (partie 7)
Organisationnelle	Développement par une seule personne	Planification rigoureuse, périmètre maîtrisé
Organisationnelle	Délai imposé par le calendrier de la formation	Priorisation des fonctionnalités (MVP)
Budgétaire	Aucun budget pour des outils ou services payants	Offres gratuites pour l'hébergement (GitHub Pages, Render)

1.6 Livrables attendus

Livrable	Description
Application web	Plateforme ITConnect : front-end React et API REST Node.js/Express connectés
Base de données	Schéma relationnel MySQL (script `itconnect.sql`) avec jeu de données de démonstration
Documentation technique	README décrivant installation, variables d'environnement, structure et routes
Dossier de projet	Le présent document, support de l'évaluation et de la soutenance

Partie 2 — Analyse et conception

2.1 Recueil et analyse du besoin

L'analyse a débuté par la rédaction de **user stories** (« En tant que… je veux… afin de… ») pour chaque type d'utilisateur, afin de constituer le backlog et d'identifier les fonctionnalités prioritaires avant toute conception technique.

- En tant que **particulier**, je veux rechercher un professionnel par compétence afin de trouver rapidement une solution à mon problème.
- En tant que **particulier**, je veux filtrer les professionnels par note afin de choisir le plus fiable.
- En tant qu'**entreprise**, je veux suivre l'historique de mes demandes afin de garder une trace de mes prestataires.
- En tant que **professionnel**, je veux compléter mon profil afin d'augmenter ma visibilité.
- En tant que **professionnel**, je veux gérer les demandes reçues afin d'organiser mon activité.
- En tant qu'**administrateur**, je veux suspendre un compte afin de garantir la sécurité de la plateforme (version ultérieure).

2.2 Spécifications fonctionnelles

Module	Fonctionnalités	Rôles concernés
Authentification	Inscription, connexion, consultation et modification du compte, changement de mot de passe, suppression du compte	Tous
Recherche	Recherche de professionnels par spécialité, ville, disponibilité, note minimale ; affichage en grille ou en liste	Tous (public)
Profil professionnel	Création et mise à jour du profil : spécialité, bio, tarif, expérience, compétences, disponibilité	Professionnel
Demandes	Création, consultation, changement de statut selon des transitions valides	Particulier, Entreprise, Professionnel
Avis	Dépôt d'un avis (note de 1 à 5 et commentaire), un seul par client et par professionnel	Particulier, Entreprise
Tableau de bord	Vue synthétique adaptée au rôle (demandes, indicateurs, activité récente)	Tous (connectés)
Entreprises partenaires	Liste des entreprises avec filtre par ville, nombre de demandes et d'avis	Public

2.3 Matrice des droits par rôle

Les droits sont appliqués **côté serveur** par deux middlewares : `authentifier` (identité vérifiée à partir du jeton) et `autoriser(...roles)` (rôle requis). Le contrôle côté interface n'est qu'un confort d'affichage.

Action	Particulier	Entreprise	Professionnel	Admin
Consulter les profils professionnels	Oui	Oui	Oui	Oui
Créer une demande	Oui	Oui	Non	Non
Répondre à une demande / changer son statut	Non	Non	Oui (destinataire)	Non
Déposer un avis	Oui	Oui	Non	Non

Action	Particulier	Entreprise	Professionnel	Admin
Modifier ou supprimer son propre compte	Oui	Oui	Oui	Oui
Modérer un compte (prévu)	Non	Non	Non	Oui

2.4 Cas d'utilisation

Acteur	Cas d'utilisation principaux
Visiteur	Consulter l'accueil, rechercher des professionnels, consulter un profil, s'inscrire, se connecter
Particulier / Entreprise	Déposer une demande, suivre ses demandes, laisser un avis, gérer son compte
Professionnel	Gérer son profil, consulter les demandes reçues, mettre à jour leur statut, consulter ses avis
Administrateur	Se connecter ; modération des comptes, avis et demandes litigieuses (version ultérieure)
Système	Générer et vérifier les jetons JWT, recalculer la note moyenne d'un professionnel

2.5 Architecture technique

L'architecture est une **architecture web en trois couches** avec séparation stricte entre le client (React), le serveur d'API (Node.js/Express) et la base de données (MySQL). Le client et le serveur communiquent par une **API REST** en JSON ; l'identité de l'utilisateur circule dans un **jeton JWT** envoyé dans l'en-tête `Authorization`.

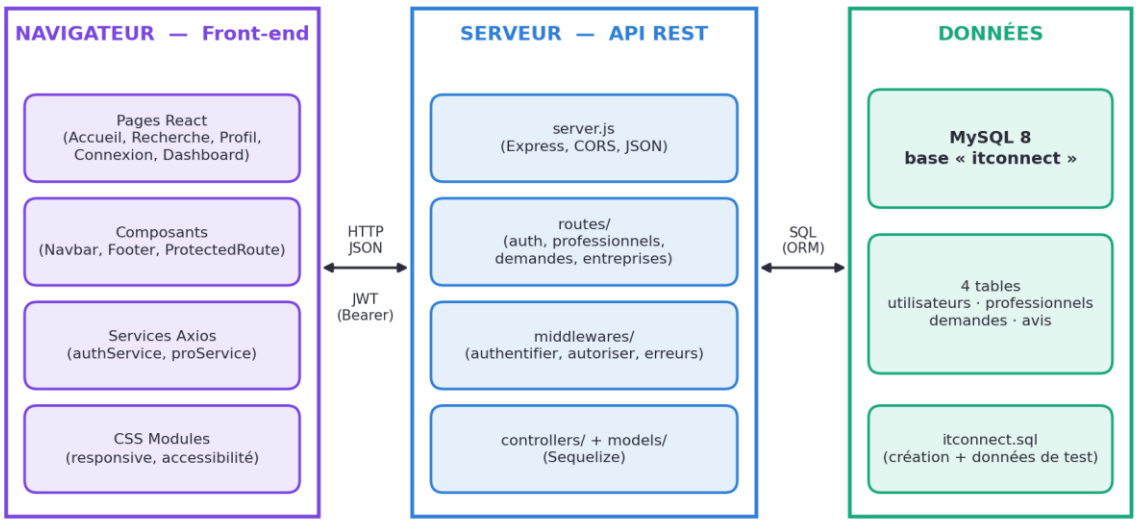


Figure 1 — Architecture technique d'ITConnect (client, API, base de données)

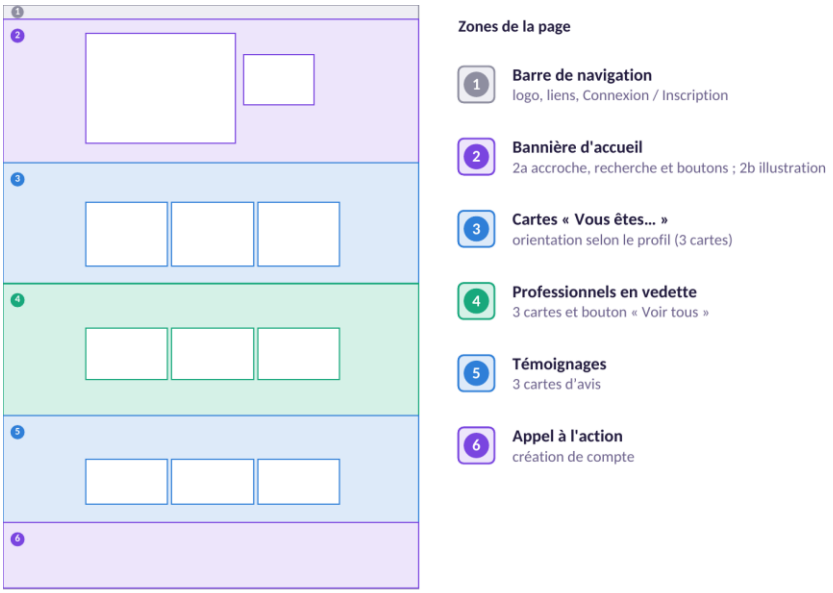
- **Couche présentation** : application monopage React (Vite), composants réutilisables, styles isolés par CSS Modules.
- **Couche métier** : API Express organisée en routes, contrôleurs et middlewares ; gestion centralisée des erreurs.
- **Couche données** : base MySQL accédée exclusivement par l'ORM Sequelize.
- **Sécurité transversale** : authentification JWT, contrôle d'accès par rôle, CORS restreint, mots de passe hachés.

2.6 Justification des choix technologiques

Besoin	Choix retenu	Alternatives	Justification
Interface	React + Vite	Vue.js, Angular	Écosystème riche, composants réutilisables, démarrage et rechargement rapides avec Vite
Serveur	Node.js + Express 4	Django, Laravel	JavaScript de bout en bout, mise en œuvre rapide d'une API REST
Base de données	MySQL 8	PostgreSQL, MongoDB	Données fortement relationnelles, large communauté, outils (Workbench, phpMyAdmin)
ORM	Sequelize 6	Prisma, TypeORM	Intégration MySQL/Express, requêtes paramétrées, associations déclaratives
Authentification	JWT + bcryptjs	Sessions serveur	Sans état côté serveur, adapté à une API REST consommée par une SPA
Appels HTTP	Axios	fetch	Instances configurées et intercepteurs de requête et de réponse
Maquettage	Figma	Adobe XD	Auto Layout, composants, prototype cliquable, export depuis l'application

2.7 Maquettage et charte graphique

Les interfaces ont été **analysées avant d'être maquettées** : le zoning découpe chaque page en grandes zones fonctionnelles hiérarchisées, puis les wireframes détaillent le contenu en niveaux de gris (images, blocs de texte, champs, boutons, cartes). Les pages développées en React ont ensuite été exportées vers Figma (extension html.to.design) et restructurées avec l'Auto Layout, des styles de texte et de couleur et des composants, afin d'obtenir des maquettes propres, modifiables et documentées.



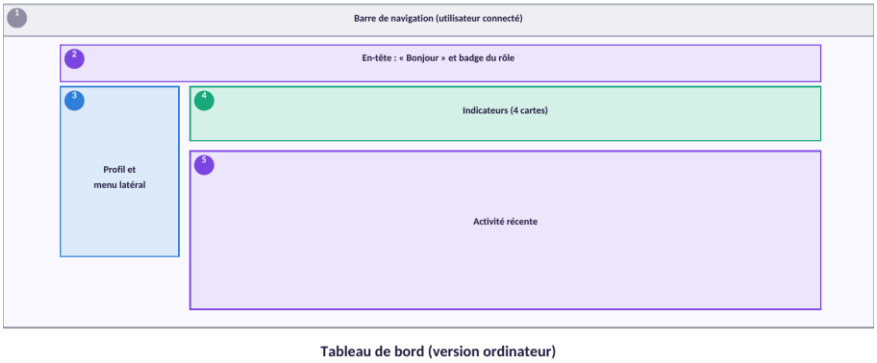


Figure 2 — Zoning de la page d'accueil et du tableau de bord (version ordinateur)

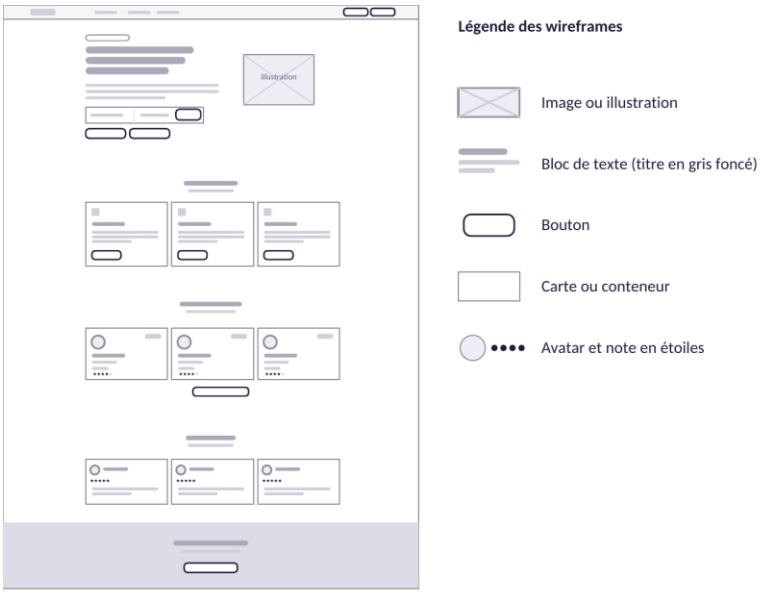


Figure 3 — Wireframes de la page d'accueil et du tableau de bord (version ordinateur)

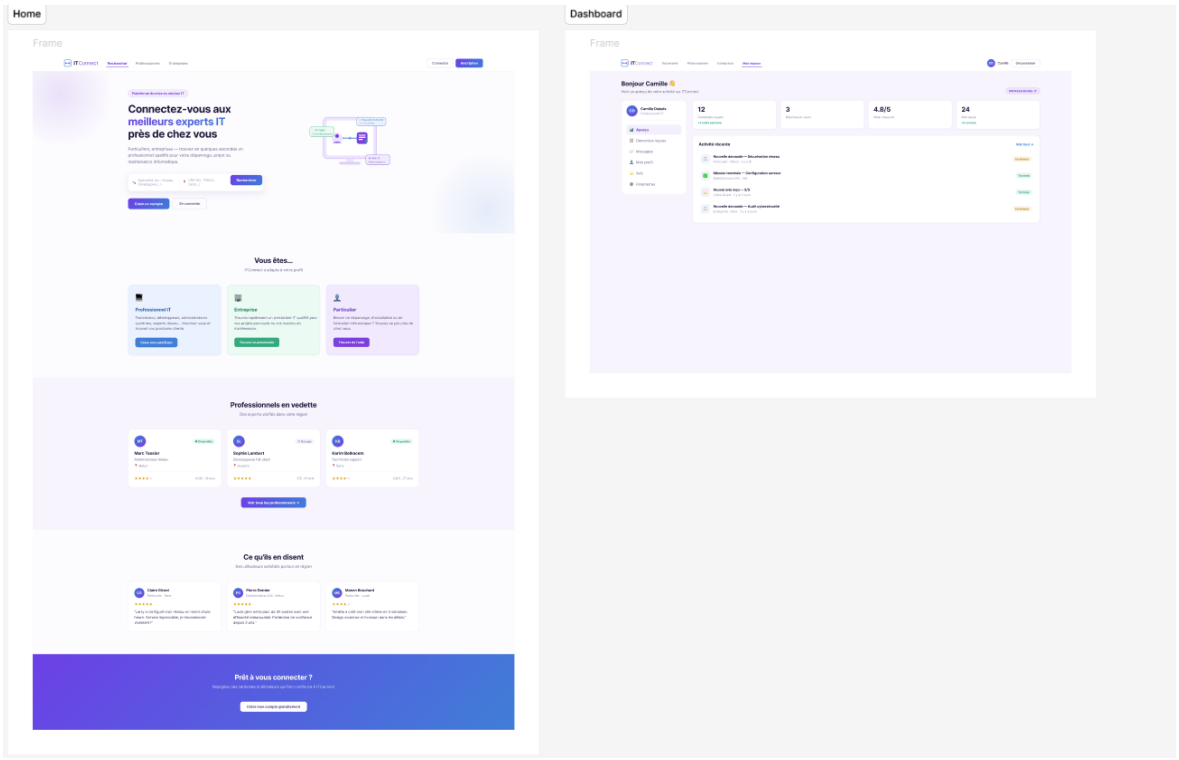


Figure 4 — Maquettes Figma : page d'accueil (Home) et tableau de bord (Dashboard)

La charte retient une palette **violet / bleu / vert** associée à une identité moderne et professionnelle. Quatre principes ont guidé la conception : **lisibilité** (contrastes, tailles de police), **simplicité** (une action principale par écran), **cohérence** (mêmes composants et emplacements) et **adaptabilité** (ordinateur, tablette, mobile).

Couleur	Code	Usage
Violet principal	#7A45E0	Éléments d'action, identité de marque
Bleu secondaire	#2F7FD8	Liens, éléments informatifs, extrémité du dégradé
Vert accent	#18A87C	Validation, indicateurs positifs (disponibilité)

Un **prototype cliquable** relie l'accueil, la connexion et le tableau de bord dans Figma. Le parcours se déroule ainsi : depuis la page d'accueil, le bouton « Connexion » ouvre la page de connexion ; le bouton « Se connecter » mène ensuite au tableau de bord de l'utilisateur. Chaque flèche de la figure suivante correspond à une interaction définie dans le mode prototype (On click → Navigate to), avec la frame d'accueil comme point de départ du flux « Parcours utilisateur ».

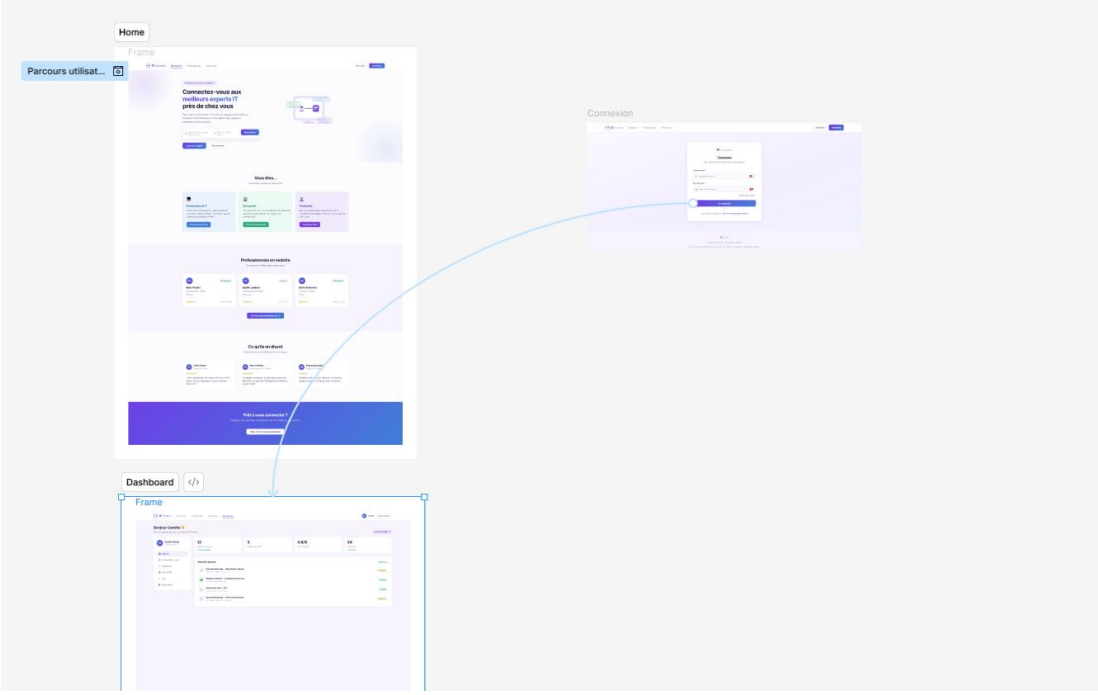


Figure 5 — Prototype Figma : liens interactifs entre l'accueil, la connexion et le tableau de bord

Prototype cliquable (parcours utilisateur) : Ouvrir le prototype ITConnect (Figma)

Fichier de maquettes (desktop, tablette, mobile) : Ouvrir le fichier de maquettes ITConnect (Figma)

Partie 3 — Conception et réalisation de la base de données

3.1 Démarche de modélisation

La modélisation suit la démarche Merise : recensement des **entités** à partir des user stories, définition des **associations** et de leurs **cardinalités** (MCD), traduction en **modèle logique relationnel** (tables, clés primaires et étrangères), puis implémentation dans MySQL (script SQL) et dans l'application (modèles Sequelize). Quatre entités structurent la plateforme : l'utilisateur, le professionnel, la demande et l'avis.

3.2 Modèle conceptuel de données (MCD)

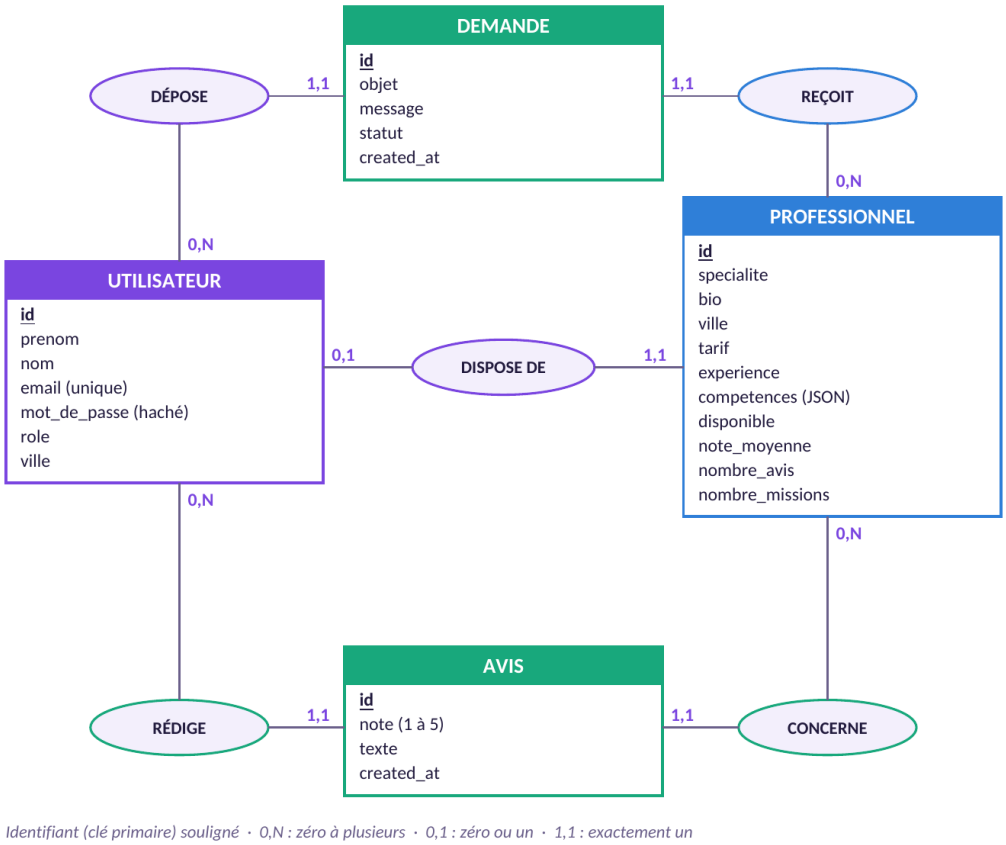


Figure 6 — MCD d'ITConnect (notation Merise)

Lecture des cardinalités (du côté de l'entité concernée) : un utilisateur **dispose de** zéro ou un profil professionnel (0,1) et un profil appartient à un seul utilisateur (1,1) ; un utilisateur **dépose** zéro à plusieurs demandes (0,N) et chaque demande est déposée par un seul utilisateur (1,1) ; un professionnel **reçoit** zéro à plusieurs demandes et chaque demande est adressée à un seul professionnel ; un utilisateur **rédige** zéro à plusieurs avis et un avis **concerne** un seul professionnel.

3.3 Diagramme de classes

Le diagramme de classes UML ci-dessous représente les mêmes entités du point de vue de l'application : chaque classe correspond à un **modèle Sequelize** et à une table. Il ajoute les types, les contraintes et les opérations utiles du modèle `User`.

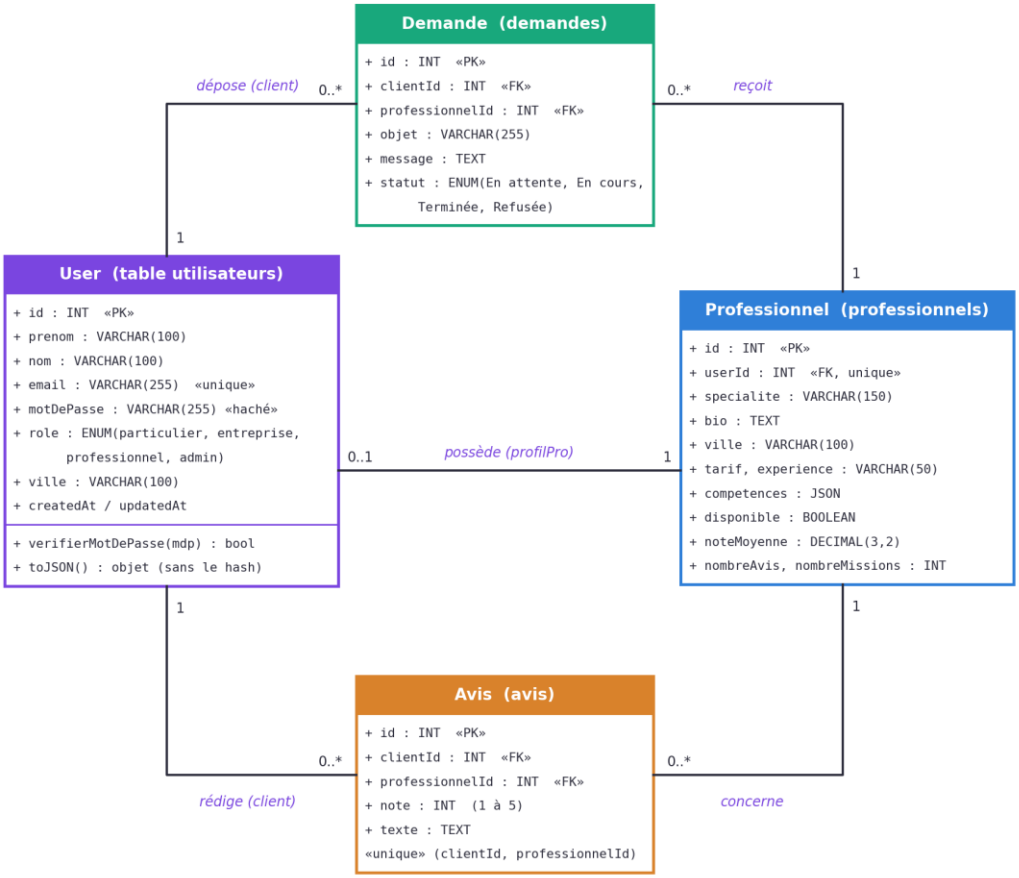


Figure 7 — Diagramme de classes UML (modèles Sequelize et tables MySQL)

> **Correspondance des noms** : les attributs des modèles Sequelize sont en camelCase (`userId`, `motDePasse`, `noteMoyenne`) ; les colonnes MySQL sont en snake_case (`user_id`, `mot_de_passe`, `note_moyenne`), la correspondance étant déclarée par l'option `field` des modèles.

3.4 Modèle logique relationnel

Table	Clé primaire	Clés étrangères	Contraintes notables
utilisateurs	id (auto-incrément)	—	email UNIQUE ; role ENUM (particulier par défaut)
professionnels	id	user_id → utilisateurs(id)	user_id UNIQUE (relation 1-1) ; disponible = 1 par défaut
demandes	id	client_id → utilisateurs(id) professionnel_id → professionnels(id)	statut ENUM (En attente par défaut)
avis	id	client_id → utilisateurs(id) professionnel_id → professionnels(id)	note entre 1 et 5 (CHECK) ; couple (client_id, professionnel_id) UNIQUE

Toutes les clés étrangères sont déclarées avec **ON DELETE CASCADE** : la suppression d'un compte supprime son profil professionnel, ses demandes et ses avis, ce qui répond au droit à l'effacement (partie 7).

3.5 Dictionnaire de données

3.5.1 Table utilisateurs

Champ	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique
prenom, nom	VARCHAR(100)	NOT NULL	Identité
email	VARCHAR(255)	NOT NULL, UNIQUE	Identifiant de connexion
mot_de_passe	VARCHAR(255)	NOT NULL	Hash bcryptjs (jamais en clair)
role	ENUM	NOT NULL, défaut particulier	particulier, entreprise, professionnel, admin
ville	VARCHAR(100)	NULL autorisé	Localisation
created_at, updated_at	DATETIME	défaut CURRENT_TIMESTAMP	Horodatage automatique

3.5.2 Table professionnels

Champ	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant du profil
user_id	INT	FK, UNIQUE, NOT NULL	Compte associé
specialite	VARCHAR(150)	NOT NULL	Domaine principal
bio	TEXT	NULL	Présentation libre
ville	VARCHAR(100)	NOT NULL	Ville d'exercice
tarif, experience	VARCHAR(50)	NULL	Tarif indicatif (ex. 65 €/h), ancienneté
competences	JSON	NULL	Liste de compétences affichées
disponible	TINYINT(1)	défaut 1	Disponibilité
note_moyenne	DECIMAL(3,2)	NULL	Moyenne des avis reçus
nombre_avis, nombre_missions	INT	défaut 0	Compteurs affichés sur le profil

3.5.3 Tables demandes et avis

Table	Champ	Type	Contraintes / description
demandes	client_id	INT	FK utilisateurs — auteur de la demande
demandes	professionnel_id	INT	FK professionnels — destinataire
demandes	objet	VARCHAR(255)	NOT NULL — objet de la demande
demandes	message	TEXT	NOT NULL — détail du besoin
demandes	statut	ENUM	En attente (défaut), En cours, Terminée, Refusée
avis	client_id	INT	FK utilisateurs — auteur de l'avis
avis	professionnel_id	INT	FK professionnels — professionnel évalué
avis	note	INT	NOT NULL, CHECK entre 1 et 5
avis	texte	TEXT	NOT NULL — commentaire
avis	(client_id, professionnel_id)	UNIQUE	Un seul avis par couple client / professionnel

Chaque table comporte en outre `created_at` et `updated_at` (horodatage automatique).

3.6 Relations et règles de gestion

- Un utilisateur possède **au plus un** profil professionnel ; un profil appartient à **un seul** utilisateur (relation 1-1 via `user_id` unique).
- Un utilisateur (particulier ou entreprise) peut déposer **plusieurs** demandes ; chaque demande a **un** auteur et **un** professionnel destinataire.
- Un avis est rédigé par **un** utilisateur et concerne **un** professionnel ; **un seul avis** par couple client / professionnel (contrainte d'unicité).
- La **note moyenne** et le **nombre d'avis** d'un professionnel sont recalculés à chaque nouvel avis.
- La suppression d'un utilisateur supprime **en cascade** son profil, ses demandes et ses avis.

Ces relations sont déclarées côté application par les **associations Sequelize**, qui permettent d'inclure les données liées dans les requêtes (par exemple le nom de l'utilisateur dans la carte d'un professionnel) :

Extrait — models/index.js : associations entre les quatre modèles

```
User.hasOne(Professionnel,  { foreignKey: 'user_id', as: 'profilPro' })
Professionnel.belongsTo(User, { foreignKey: 'user_id', as: 'user' })

User.hasMany(Demande, { foreignKey: 'client_id', as: 'demandesEnvoyees' })
Demande.belongsTo(User, { foreignKey: 'client_id', as: 'client' })

Professionnel.hasMany(Demande, { foreignKey: 'professionnel_id', as: 'demandesRecues' })
Demande.belongsTo(Professionnel, { foreignKey: 'professionnel_id', as: 'professionnel' })

User.hasMany(Avis, { foreignKey: 'client_id', as: 'avisLaisses' })
Avis.belongsTo(User, { foreignKey: 'client_id', as: 'client' })

Professionnel.hasMany(Avis, { foreignKey: 'professionnel_id', as: 'avis' })
Avis.belongsTo(Professionnel, { foreignKey: 'professionnel_id', as: 'professionnel' })
```

3.7 Création, configuration et données de démonstration

La base est créée par le script **`itconnect.sql`**, exécutable dans phpMyAdmin ou MySQL Workbench : création de la base en `utf8mb4`, des quatre tables avec leurs contraintes, puis insertion d'un **jeu de démonstration** (17 comptes, 7 profils professionnels, 6 demandes, 5 avis) et d'une requête de vérification des effectifs. Extrait pour la table des avis :

Extrait — itconnect.sql : table avis

```
CREATE TABLE IF NOT EXISTS avis (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  client_id       INT           NOT NULL,
  professionnel_id INT          NOT NULL,
  note            INT           NOT NULL CHECK (note BETWEEN 1 AND 5),
  texte           TEXT          NOT NULL,
  created_at      DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at      DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uq_avis (client_id, professionnel_id),
  FOREIGN KEY (client_id)       REFERENCES utilisateurs(id)   ON DELETE CASCADE,
  FOREIGN KEY (professionnel_id) REFERENCES professionnels(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Côté application, la connexion est configurée dans `config/database.js` : instance Sequelize en dialecte MySQL, paramètres lus dans les **variables d'environnement** (dotenv) pour qu'aucun identifiant ne figure dans le code, jeu de caractères `utf8mb4`, horodatage automatique et fonction `connectDB()` qui teste la connexion et arrête le processus en cas d'échec.

backend > config > JS database.js > sequelize > define

```
const { Sequelize } = require('sequelize')
require('dotenv').config()

const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host:    process.env.DB_HOST || 'localhost',
    port:    process.env.DB_PORT || 3306,
    dialect: 'mysql',
    logging: false,
    define: {
      charset:    'utf8mb4',
      collate:    'utf8mb4_unicode_ci',
      timestamps: true,
    },
  }
)

async function connectDB() {
  try {
    await sequelize.authenticate()
    console.log('✅ Connexion MySQL établie avec succès.')
  } catch (error) {
    console.error('❌ Impossible de se connecter à MySQL :', error.message)
    process.exit(1)
  }
}

module.exports = { sequelize, connectDB }
```

Figure 8 — Fichier Backend/config/database.js

Au démarrage, `server.js` appelle `sequelize.sync({ force: false })` : les tables manquantes sont créées, mais **les tables et données existantes ne sont jamais supprimées**. Le script SQL reste la référence du schéma.

Partie 4 — Réalisation technique

4.1 Environnement et outils de développement

Catégorie	Outils et technologies
Front-end	React, Vite, React Router, CSS Modules, Axios
Back-end	Node.js (≥ 18), Express 4, Sequelize 6, mysql2, jsonwebtoken, bcryptjs, cors, dotenv, nodemon
Base de données	MySQL 8, MySQL Workbench, phpMyAdmin (XAMPP)
Conception	Figma, extension html.to.design
Tests	Postman (tests de l'API), Chrome DevTools, Jest (tests unitaires)
Versionnement	Git, GitHub (branches `feature/dev` et `main`)
Éditeur	Visual Studio Code

4.2 Organisation du code

Le projet est séparé en deux dossiers, `Backend` et `Frontend`, qui peuvent être développés, testés et déployés indépendamment.

Arborescence du projet

```
ITConnect/
├── Backend/
│   ├── config/database.js          # connexion Sequelize / MySQL
│   ├── controllers/                # logique métier (auth, demandes, professionnels…)
│   ├── middlewares/
│   │   ├── auth.js                 # authentifier, autoriser
│   │   └── erreurs.js              # gestion centralisée des erreurs
│   ├── models/                     # User, Professionnel, Demande, Avis, index.js
│   ├── routes/                     # auth, professionnels, demandes, entreprises
│   ├── itconnect.sql               # création de la base + données de démonstration
│   ├── server.js                   # point d'entrée Express
│   ├── .env                        # secrets (non versionné)
│   └── package.json
└── Frontend/
    └── src/
        ├── components/             # Navbar, Footer, ProtectedRoute
        ├── pages/                  # Home, Recherche, Profil, Connexion, Inscription, Dashboard…
        └── services/               # authService, proService (Axios)
```

4.3 Réalisation du back-end

4.3.1 Serveur Express

`server.js` charge les variables d'environnement, configure **CORS** (origine limitée à l'adresse du front-end), le décodage JSON, l'en-tête UTF-8, monte les quatre groupes de routes sous `/api`, renvoie une **404 JSON** pour toute route inconnue puis branche le middleware d'erreurs. Il se connecte à MySQL, synchronise les modèles et démarre l'écoute sur le port défini par `PORT` (3000 par défaut).

Extrait — server.js : CORS, routes, 404 et gestion des erreurs

```
app.use(cors({
  origin:      process.env.FRONTEND_URL || 'http://localhost:5173',
  credentials: true,
}))
app.use(express.json())

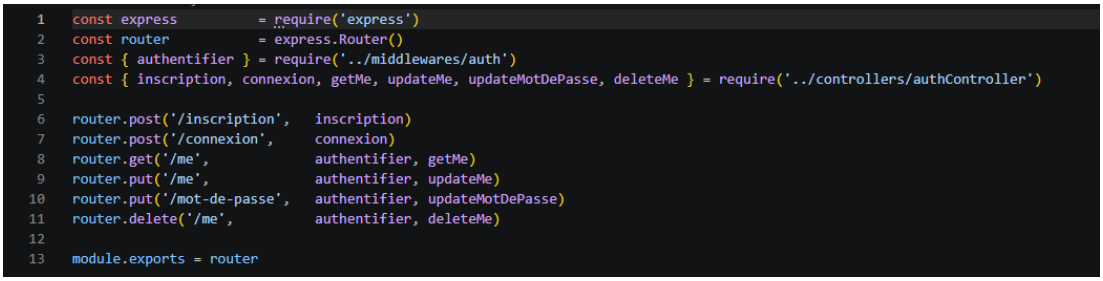
app.use('/api/auth',           authRoutes)
app.use('/api/professionnels', professionnelsRoutes)
app.use('/api/demandes',       demandesRoutes)
```

```
app.use('/api/entreprises',    entreprisesRoutes)

app.use((req, res) => {
  res.status(404).json({ message: `Route ${req.method} ${req.path} introuvable.` })
})
app.use(gestionErreurs)
```

4.3.2 Routes de l'API

Chaque ressource dispose de son fichier de routes ; les routes sensibles sont protégées par `authentifier` (et `autoriser` lorsque le rôle compte). Index complet en annexe A.



```
const express          = require('express')
const router           = express.Router()
const { authentifier } = require('../middlewares/auth')
const { inscription, connexion, getMe, updateMe, updateMotDePasse, deleteMe } = require('../controllers/authController')

router.post('/inscription',   inscription)
router.post('/connexion',     connexion)
router.get('/me',             authentifier, getMe)
router.put('/me',             authentifier, updateMe)
router.put('/mot-de-passe',   authentifier, updateMotDePasse)
router.delete('/me',          authentifier, deleteMe)

module.exports = router
```

Figure 9 — Fichier Backend/routes/auth.js : routes d'authentification

Méthode	Route	Description	Accès
POST	/api/auth/inscription	Création d'un compte	Public
POST	/api/auth/connexion	Connexion, génération du jeton	Public
GET / PUT	/api/auth/me	Consulter / modifier son compte	Authentifié
PUT	/api/auth/mot-de-passe	Changer son mot de passe	Authentifié
DELETE	/api/auth/me	Supprimer son compte (droit à l'effacement)	Authentifié
GET	/api/professionnels (/:id)	Recherche avec filtres, détail d'un profil	Public
POST	/api/demandes	Créer une demande	Particulier, Entreprise
PUT	/api/demandes/:id/statut	Changer le statut d'une demande	Professionnel

4.3.3 Contrôleurs

Les contrôleurs contiennent la logique métier. Celui de l'authentification (`authController.js`) gère l'inscription, la connexion et le compte courant. L'extrait ci-dessous montre l'inscription : **liste blanche des rôles**, contrôle du format du mot de passe, vérification de l'unicité de l'e-mail (409), création automatique du profil pour un professionnel et émission du jeton.

Extrait — controllers/authController.js : inscription sécurisée

```
const ROLES_AUTORISES = ['particulier', 'entreprise', 'professionnel']
const REGEX_MOT_DE_PASSE = /^(?=.*[A-Z])(?=.*\d).{8,}$/

async function inscription(req, res, next) {
  try {
    const { prenom, nom, email, motDePasse, role } = req.body
    if (!prenom || !nom || !email || !motDePasse)
      return res.status(400).json({ message: 'Tous les champs sont obligatoires.' })
    if (!REGEX_MOT_DE_PASSE.test(motDePasse))
      return res.status(400).json({ message: 'Mot de passe : 8 caractères min., 1 majuscule, 1 chiffre.' })

    const roleFinal = ROLES_AUTORISES.includes(role) ? role : 'particulier'   // jamais « admin »
    const existant = await User.findOne({ where: { email } })
    if (existant) return res.status(409).json({ message: 'Email déjà utilisé.' })

    const user = await User.create({ prenom, nom, email, motDePasse, role: roleFinal })
    if (roleFinal === 'professionnel')
      await Professionnel.create({ userId: user.id, specialite: 'À compléter', ville: '' })
```

```
    const token = genererToken(user.id)
    res.status(201).json({ message: 'Compte créé.', token, user: user.toJSON() })
  } catch (error) { next(error) }
}
```

La création d'une demande illustre le principe « ne jamais faire confiance au corps de la requête » : seuls `professionnelId`, `objet` et `message` sont lus ; l'auteur provient du **jeton** (`req.user.id`) et le statut prend sa **valeur par défaut**.

Extrait — controllers/demandeController.js : création d'une demande

```
async function createDemande(req, res, next) {
  try {
    const { professionnelId, objet, message } = req.body
    const pro = await Professionnel.findByPk(professionnelId)
    if (!pro) return res.status(404).json({ message: 'Professionnel introuvable.' })
    if (pro.userId === req.user.id)
      return res.status(400).json({ message: 'Vous ne pouvez pas vous envoyer une demande.' })
    const demande = await Demande.create({ clientId: req.user.id, professionnelId, objet, message })
    res.status(201).json({ message: 'Demande envoyée.', demande })
  } catch (error) { next(error) }
}
```

4.3.4 Middlewares : authentification, autorisation, erreurs

`authentifier` extrait le jeton de l'en-tête `Authorization: Bearer …`, le vérifie avec `JWT_SECRET` et **relit l'utilisateur en base** : un compte supprimé est donc refusé immédiatement. `autoriser(...roles)` répond **403** si le rôle n'est pas autorisé. `gestionErreurs` convertit les erreurs Sequelize en réponses HTTP cohérentes (400 validation, 409 doublon, 401 jeton, 500 sans détail interne).



```
const jwt       = require('jsonwebtoken')
const { User }  = require('../models')

async function authentifier(req, res, next) {
  try {
    const authHeader = req.headers.authorization
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ message: 'Token manquant.' })
    }
    const token   = authHeader.split(' ')[1]
    const decoded = jwt.verify(token, process.env.JWT_SECRET)
    const user    = await User.findByPk(decoded.id)
    if (!user) return res.status(401).json({ message: 'Utilisateur introuvable.' })
    req.user = user
    next()
  } catch (error) {
    if (error.name === 'TokenExpiredError')
      return res.status(401).json({ message: 'Session expirée.' })
    return res.status(401).json({ message: 'Token invalide.' })
  }
}

function autoriser(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role))
      return res.status(403).json({ message: 'Accès interdit.' })
    next()
  }
}

module.exports = { authentifier, autoriser }
```

Figure 10 — Fichier Backend/middlewares/auth.js : authentifier et autoriser

```
function gestionErreurs(err, req, res, next) {
  console.error('❌ Erreur :', err.message)
  if (err.name === 'SequelizeValidationError')
    return res.status(400).json({ message: err.errors[0].message })
  if (err.name === 'SequelizeUniqueConstraintError')
    return res.status(409).json({ message: 'Cette valeur est déjà utilisée.' })
  if (err.name === 'JsonWebTokenError')
    return res.status(401).json({ message: 'Token invalide.' })
  if (err.statusCode)
    return res.status(err.statusCode).json({ message: err.message })
  res.status(500).json({ message: 'Erreur interne du serveur.' })
}

module.exports = gestionErreurs
```

Figure 11 — Fichier Backend/middlewares/erreurs.js : gestion centralisée des erreurs

4.3.5 Dépendances

```
{
  "name": "itconnect-backend",
  "version": "1.0.0",
  "description": "API REST ITConnect - Plateforme de mise en relation IT",
  "main": "server.js",
  ▷Debug
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2",
    "mysql2": "^3.9.7",
    "sequelize": "^6.37.3"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

Figure 12 — Fichier Backend/package.json : dépendances et scripts (start, dev)

4.4 Réalisation du front-end

Le front-end est une application monopage **React** (Vite). Les pages correspondent aux parcours principaux : accueil, recherche, profil professionnel, connexion, inscription, entreprises partenaires et tableau de bord multi-rôles. Les composants communs (`Navbar`, `Footer`) encadrent toutes les pages ; `ProtectedRoute` redirige vers `/connexion` un visiteur non authentifié. Les styles sont isolés par **CSS Modules**.

Tous les appels réseau passent par deux **services Axios** (`authService` pour le compte, `proService` pour les professionnels, demandes et avis). Chaque instance ajoute automatiquement le jeton à chaque requête et un **intercepteur de réponse** nettoie la session et redirige vers la connexion en cas d'erreur 401 (jeton expiré ou invalide).

Extrait — services/authService.js : instance Axios et intercepteurs

```
const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:3000'

const api = axios.create({
  baseURL: API_URL,
  headers: { 'Content-Type': 'application/json' },
})

api.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token'); localStorage.removeItem('user')
      window.location.href = '/connexion'
```

```
    }
    return Promise.reject(error)
  }
)

export const authService = {
  login:    async ({ email, motDePasse }) => (await api.post('/api/auth/connexion', { email, motDePasse
})).data,
  register: async (donnees)               => (await api.post('/api/auth/inscription', donnees)).data,
  getMe:    async ()                      => (await api.get('/api/auth/me')).data,
}
```

4.5 Fonctionnement global : lien entre le front-end et le back-end

Le front-end et le back-end sont deux applications indépendantes qui ne partagent que le **contrat de l'API** (routes, formats JSON, codes HTTP). Le parcours type d'un utilisateur illustre leur coopération : (1) l'utilisateur remplit un formulaire React ; (2) le composant appelle une fonction du **service Axios** ; (3) la requête HTTP part vers l'API avec, si l'utilisateur est connecté, le jeton JWT ; (4) Express route la requête, `authentifier` / `autoriser` contrôlent l'accès ; (5) le **contrôleur** interroge MySQL via **Sequelize** ; (6) la réponse JSON revient au composant qui met à jour l'interface.

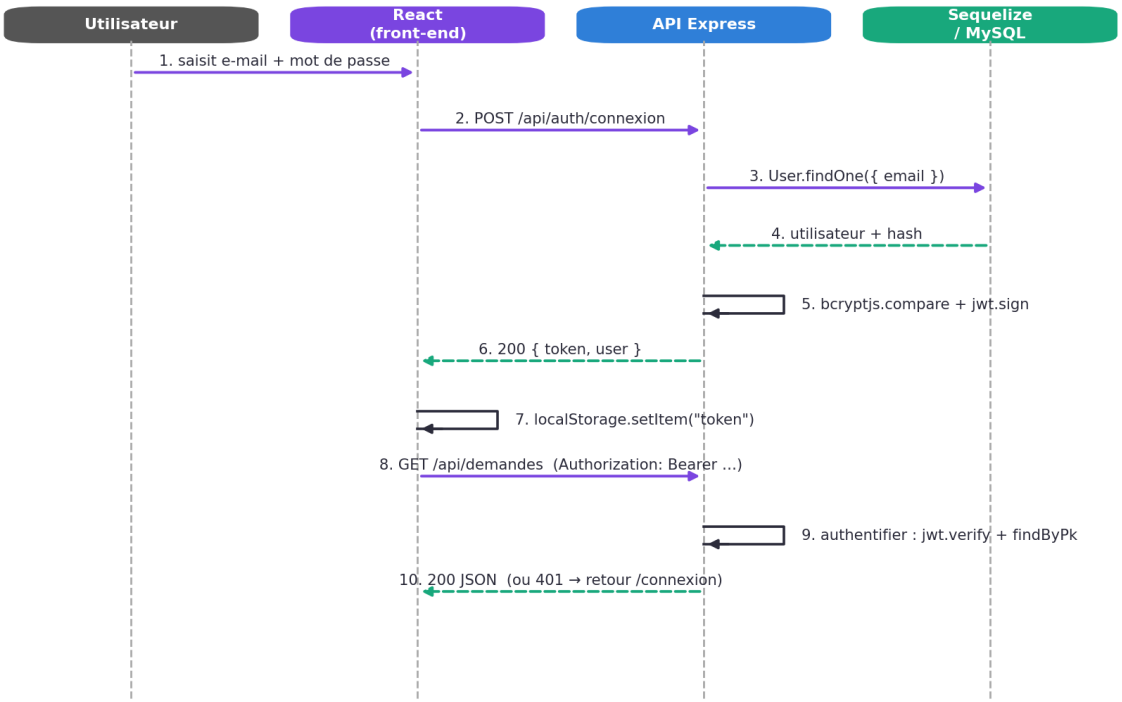


Figure 13 — Diagramme de séquence : connexion puis appel d'une route protégée

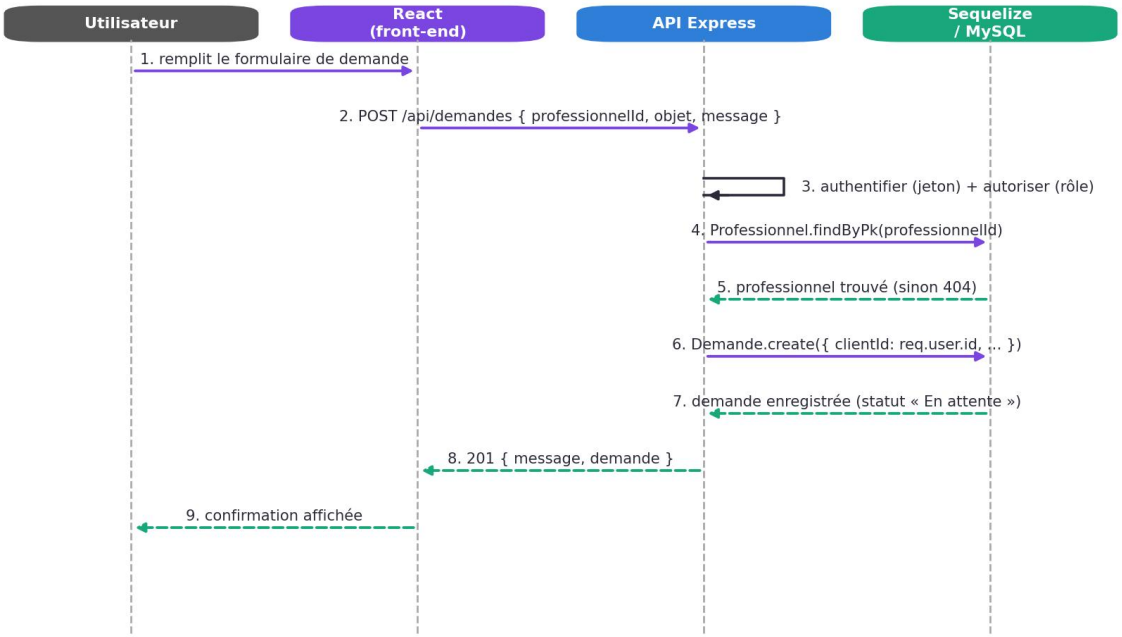


Figure 14 — Diagramme de séquence : dépôt d'une demande par un client

Action de l'utilisateur	Composant / service React	Route de l'API	Contrôles côté serveur	Table concernée
S'inscrire	Page Inscription · authService.register	POST /api/auth/inscription	Champs, mot de passe, rôle (liste blanche), e-mail unique	utilisateurs (+ professionnels)
Se connecter	Page Connexion · authService.login	POST /api/auth/connexion	Comparaison du hash, jeton signé	utilisateurs
Rechercher	Page Recherche · proService.getProfessionnels	GET /api/professionnels	Filtres (requête paramétrée)	professionnels, utilisateurs
Déposer une demande	Profil · proService.createDemande	POST /api/demandes	Jeton, rôle, professionnel existant	demandes
Laisser un avis	Profil · proService.createAvis	POST /api/professionnels/:id/avis	Jeton, rôle, un seul avis par couple	avis, professionnels
Supprimer son compte	Paramètres · authService.deleteAccount	DELETE /api/auth/me	Jeton ; suppression en cascade	toutes

4.6 Interfaces réalisées

Les captures ci-dessous montrent l'application en fonctionnement (version ordinateur, serveur de développement `localhost:5173`).

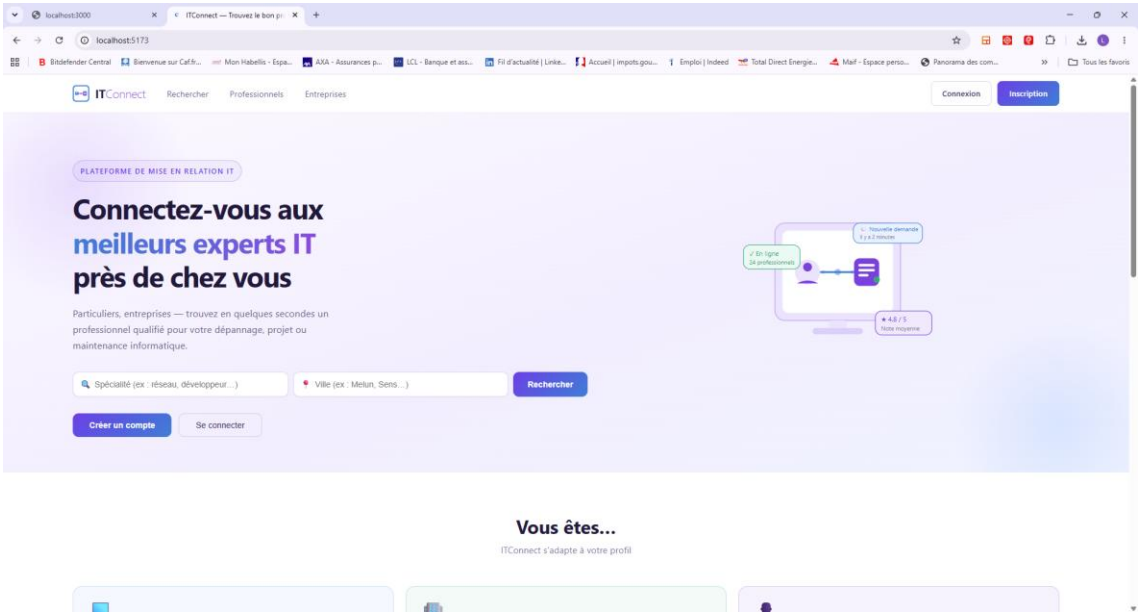


Figure 15 — Page d'accueil : bannière, recherche par spécialité et ville, accès par profil

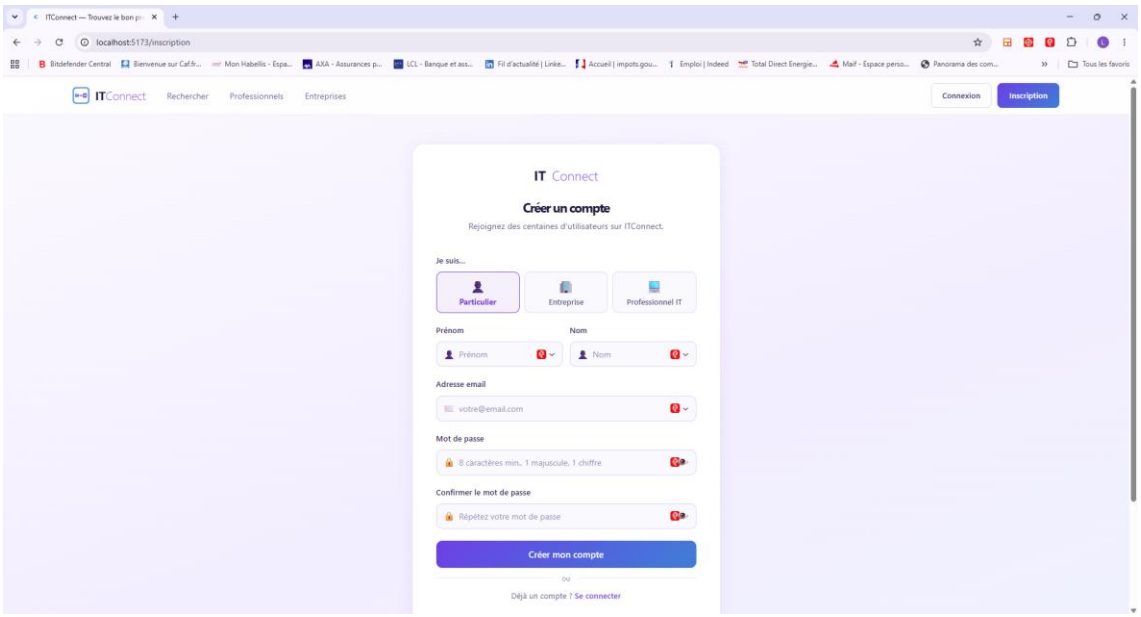


Figure 16 — Page d'inscription : choix du profil (Particulier, Entreprise, Professionnel IT)

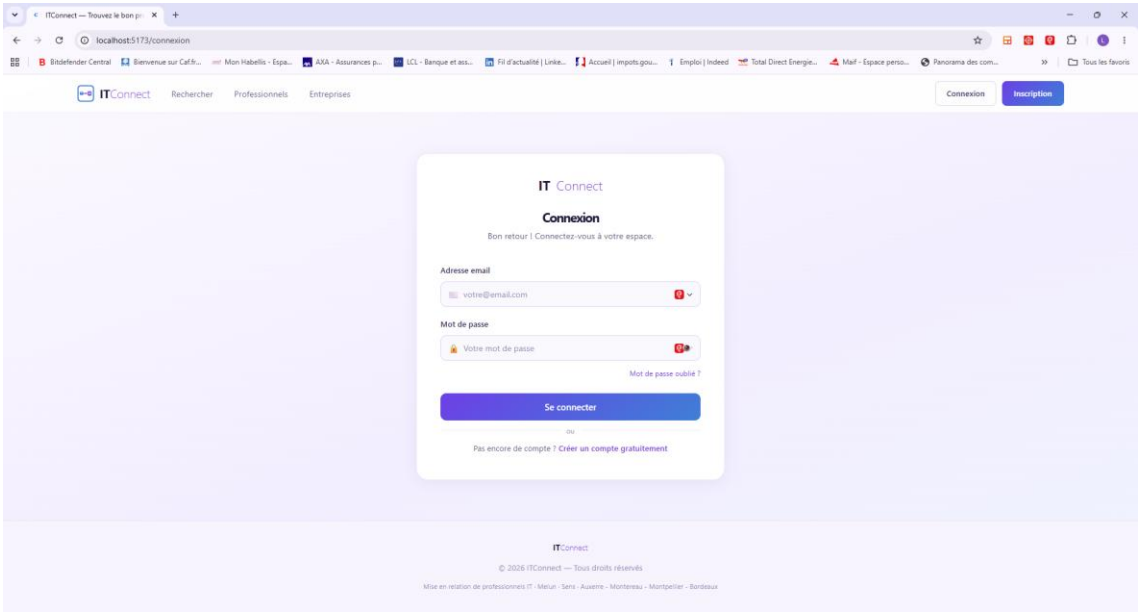


Figure 17 — Page de connexion

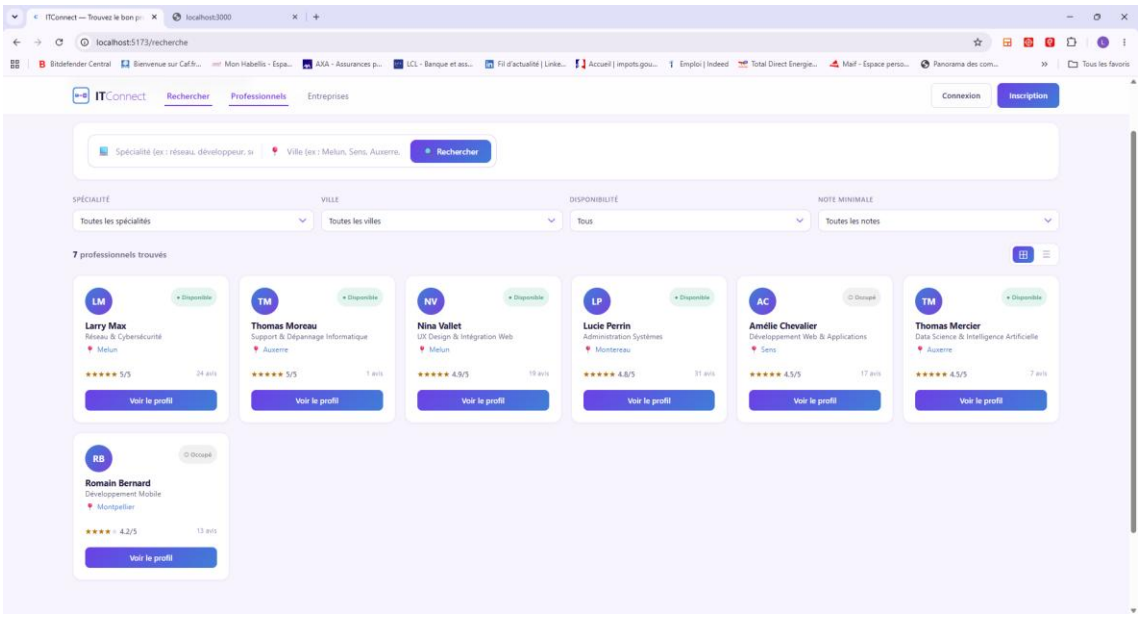


Figure 18 — Page de recherche : filtres combinables, compteur de résultats, grille ou liste

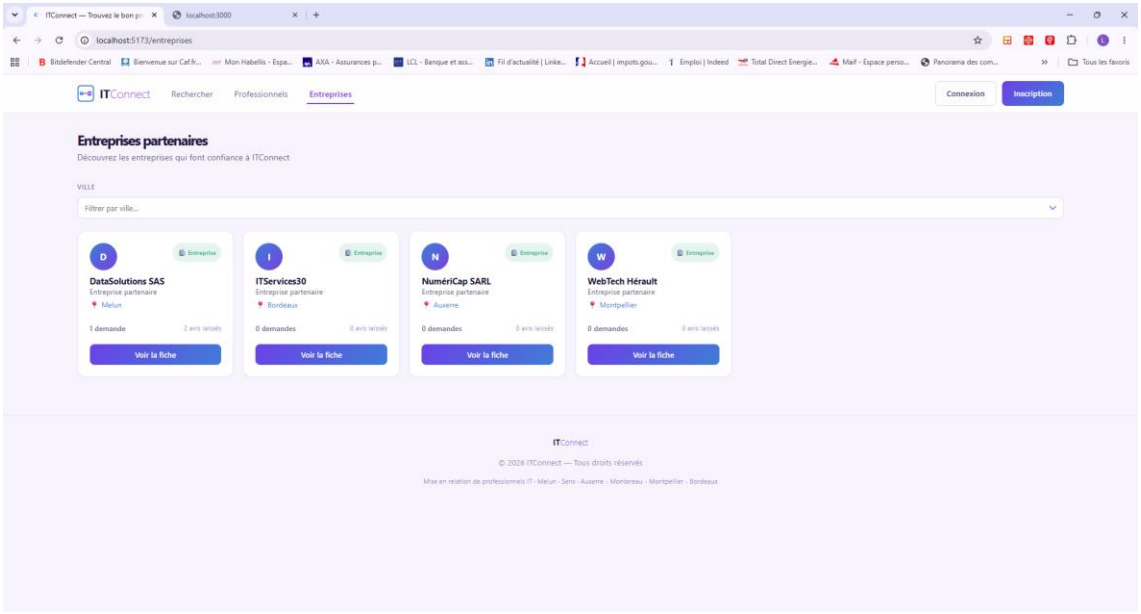


Figure 19 — Page des entreprises partenaires : filtre par ville

4.7 Interface responsive

L'interface doit rester pleinement utilisable sur smartphone, tablette et ordinateur. La mise en page repose sur des **unités relatives** (%, rem, fr), des **grilles CSS** et des **media queries** définies au plus près de chaque composant grâce aux CSS Modules.

Format	Largeur de référence	Adaptations principales
Ordinateur	1024 px et plus	Navigation horizontale complète, grilles de cartes sur 3 colonnes, bannière sur deux colonnes
Tablette	640 à 1023 px	Menu replié (burger), cartes sur 2 colonnes, formulaires centrés
Mobile	moins de 640 px	Une colonne, champs de recherche empilés, boutons en pleine largeur

Extrait — grille de cartes responsive (CSS Modules)

```
.liste {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1.5rem;
}
@media (max-width: 1023px) { .liste { grid-template-columns: repeat(2, 1fr); } }
@media (max-width: 639px)  { .liste { grid-template-columns: 1fr; } }
```



Figure 20 — Accueil, tablette



Figure 21 — Accueil, mobile

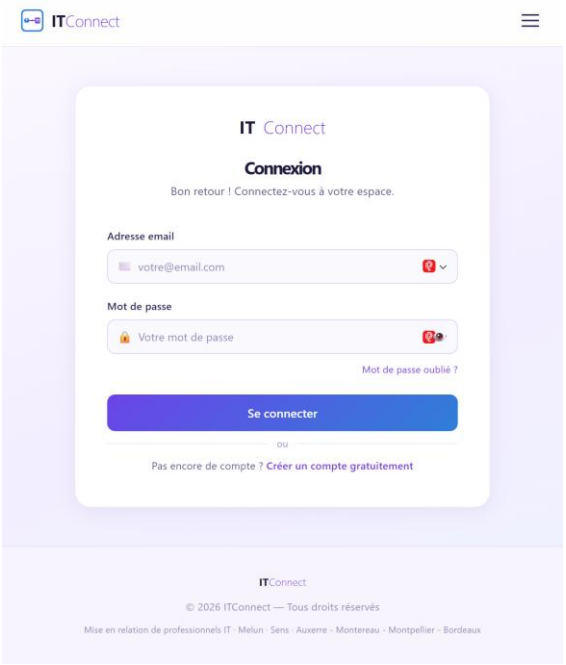


Figure 22 — Connexion, tablette

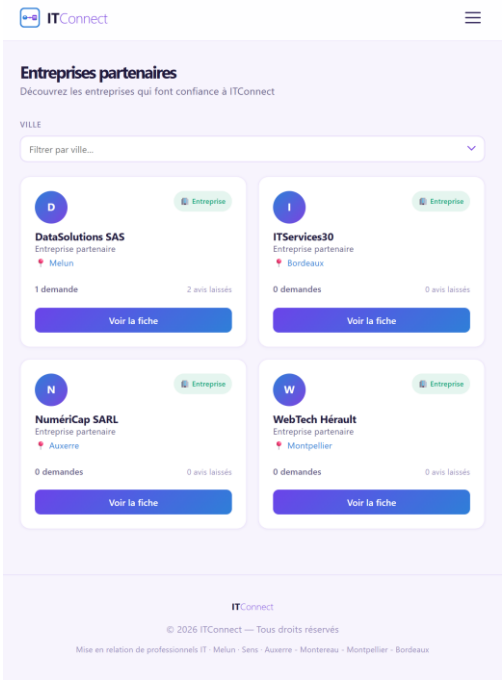


Figure 23 — Entreprises, tablette



Figure 24 — Entreprises, mobile



Figure 25 — Recherche, mobile

> **Point à améliorer identifié sur les captures** : sur mobile et tablette, l'icône de loupe du champ « Spécialité » de la bannière d'accueil s'affiche sous le champ au lieu d'être placée à l'intérieur. La correction consiste à placer l'icône en position absolue dans un conteneur en position relative (voir partie 10).

4.8 Accessibilité et référencement

- **Structure** : un seul `h1` par page, titres hiérarchisés, balises sémantiques (`nav`, `main`, `section`, `footer`).
- **Clavier et lecteurs d'écran** : éléments natifs focalisables, `aria-label` et `aria-expanded` sur le menu burger, alternative textuelle des étoiles de notation, cartes activables au clavier, éléments décoratifs masqués (`aria-hidden`).
- **Contrastes** : ratios de la palette calculés selon le WCAG ; le violet #7A45E0 (5,59:1) respecte le seuil AA, le bleu #2F7FD8 (4,08:1) est réservé au grand texte et aux éléments graphiques.
- **Référencement** : langue, viewport, `title` et `description` dans `index.html`, URLs lisibles (`/pro/12`, `/connexion`) ; titres et descriptions propres à chaque page prévus (bibliothèque dédiée).

Partie 5 — Tests et recette

5.1 Stratégie de test

La qualité de l'application est vérifiée à **quatre niveaux complémentaires**, du plus fin au plus global. Les tests s'exécutent sur un environnement local isolé : base `itconnect` alimentée par le jeu de démonstration, API sur le port 3000 et front-end sur le port 5173.

Niveau	Objet	Outil	Quand
Tests unitaires	Fonctions et middlewares isolés (règles de validation, contrôle des rôles, conversion des erreurs)	Jest	À chaque modification du code métier
Tests fonctionnels de l'API	Chaque route : codes HTTP, corps de réponse, droits d'accès, cas d'erreur	Postman	Avant chaque livraison
Recette de l'interface	Parcours utilisateur de bout en bout (inscription, recherche, demande, avis)	Navigateur, DevTools	Avant chaque livraison
Compatibilité et responsive	Chrome, Firefox, Edge ; largeurs 375, 768 et 1440 px	DevTools (mode appareil)	Fin de chaque page

5.2 Tests unitaires

Les tests unitaires ciblent le code **sans base de données** : le modèle `User` est remplacé par un double (`jest.mock`), ce qui rend les tests rapides et reproductibles. Sont couverts : le middleware de contrôle des rôles `autoriser` et la conversion des erreurs par `gestionErreurs`.

Extrait — jeu de tests unitaires (Jest) : middlewares

```
// tests/middlewares.test.js
jest.mock('../models', () => ({ User: {} }))
const { autoriser } = require('../middlewares/auth')
const gestionErreurs = require('../middlewares/erreurs')

const resFactice = () => {
  const res = {}
  res.status = jest.fn().mockReturnValue(res)
  res.json   = jest.fn().mockReturnValue(res)
  return res
}

describe('autoriser', () => {
  test('laisse passer un rôle autorisé', () => {
    const next = jest.fn()
    autoriser('professionnel')({ user: { role: 'professionnel' } }, resFactice(), next)
    expect(next).toHaveBeenCalled()
  })
  test('refuse un rôle non autorisé (403)', () => {
    const res = resFactice(), next = jest.fn()
    autoriser('professionnel')({ user: { role: 'particulier' } }, res, next)
    expect(res.status).toHaveBeenCalledWith(403)
    expect(next).not.toHaveBeenCalled()
  })
})

describe('gestionErreurs', () => {
  test('doublon SQL -> 409', () => {
    const res = resFactice()
    gestionErreurs({ name: 'SequelizeUniqueConstraintError' }, {}, res, jest.fn())
    expect(res.status).toHaveBeenCalledWith(409)
  })
  test('erreur inconnue -> 500 sans détail interne', () => {
    const res = resFactice()
    gestionErreurs(new Error('secret'), {}, res, jest.fn())
```

```
    expect(res.status).toHaveBeenCalledWith(500)
    expect(res.json).toHaveBeenCalledWith({ message: 'Erreur interne du serveur.' })
  })
})
```

Réf.	Fonction testée	Cas	Résultat attendu	Statut
TU1	autoriser	Rôle présent dans la liste	next() appelé	À exécuter
TU2	autoriser	Rôle absent de la liste	Réponse 403, next() non appelé	À exécuter
TU3	gestionErreurs	SequelizeUniqueConstraintError	Réponse 409	À exécuter
TU4	gestionErreurs	JsonWebTokenError	Réponse 401 « Token invalide. »	À exécuter
TU5	gestionErreurs	Erreur inconnue	Réponse 500 sans fuite de détail	À exécuter
TU6	Validation du mot de passe	« abc », « abcdefgh », « Abcdefg1 »	Refus, refus, acceptation	À exécuter

Exécution : `npm install --save-dev jest` puis `npx jest` dans le dossier `Backend` (script `"test": "jest"` dans `package.json`).

5.3 Tests fonctionnels de l'API avec Postman

Les routes sont testées avec **Postman** dans une collection « ITConnect API » regroupée par ressource (Auth, Professionnels, Demandes, Avis). Un environnement définit `{{baseUrl}}` (`http://localhost:3000`) et `{{token}}`. Un script de test de la requête de connexion enregistre automatiquement le jeton pour les requêtes suivantes et vérifie le code de réponse :

Extrait — script de test Postman

```
// Onglet « Tests » de la requête POST {{baseUrl}}/api/auth/connexion
pm.test('Connexion réussie (200)', () => pm.response.to.have.status(200))
pm.test('Un jeton est renvoyé', () => {
  const json = pm.response.json()
  pm.expect(json.token).to.be.a('string')
  pm.expect(json.user).to.not.have.property('motDePasse')   // le hash ne doit jamais sortir
  pm.environment.set('token', json.token)
})
```

Réf.	Requête	Résultat attendu	Statut
TF1	POST /api/auth/inscription (données valides)	201, jeton, rôle demandé	Conforme
TF2	POST /api/auth/inscription (e-mail déjà utilisé)	409 « Email déjà utilisé. », aucun compte créé	Conforme
TF3	POST /api/auth/inscription avec `"role": "admin"`	201, compte créé avec le rôle particulier	À exécuter
TF4	POST /api/auth/inscription (mot de passe « abc »)	400, message sur le format du mot de passe	À exécuter
TF5	POST /api/auth/connexion (identifiants valides)	200, jeton, utilisateur sans hash	Conforme
TF6	POST /api/auth/connexion (mot de passe erroné)	401, message générique identique à « e-mail inconnu »	Conforme
TF7	GET /api/auth/me sans jeton	401 « Token manquant. »	Conforme
TF8	GET /api/auth/me avec jeton falsifié	401 « Token invalide. »	À exécuter
TF9	GET /api/professionnels?specialite=…	200, liste filtrée (route publique)	Conforme
TF10	POST /api/demandes (client connecté)	201, statut « En attente »	Conforme
TF11	POST /api/demandes sans jeton	401	Conforme
TF12	PUT /api/demandes/:id/statut par un client	403 « Accès interdit. »	À exécuter
TF13	POST /api/professionnels/:id/avis (second avis du même client)	409 « Avis déjà soumis »	Conforme

Réf.	Requête	Résultat attendu	Statut
TF14	DELETE /api/auth/me puis requête SQL de contrôle	200 ; profil, demandes et avis supprimés en cascade	À exécuter

[À COMPLÉTER : insérer ici une capture de la collection Postman et de l'exécution (Collection Runner) montrant les tests au vert.] Les statuts « À exécuter » sont à renseigner après l'exécution de chaque test.

5.4 Recette de l'interface

Réf.	Parcours	Résultat attendu	Statut
R1	Inscription avec mot de passe non conforme	Message d'erreur explicite avant envoi, aucun appel à l'API	À exécuter
R2	Connexion puis rechargement de la page	Utilisateur toujours connecté (jeton conservé), accès au tableau de bord	À exécuter
R3	Accès à une page protégée sans être connecté	Redirection vers /connexion	À exécuter
R4	Expiration ou falsification du jeton	Session supprimée, redirection vers /connexion (intercepteur 401)	À exécuter
R5	Recherche par spécialité et ville, filtres combinés	Liste filtrée, compteur de résultats exact	Conforme
R6	Bascule grille / liste	Affichage modifié sans rechargement	À exécuter
R7	Dépôt d'une demande depuis un profil	Confirmation visible, demande présente dans le tableau de bord du professionnel	Conforme
R8	Menu burger en mobile et tablette	Ouverture et fermeture, état annoncé aux lecteurs d'écran	À exécuter

5.5 Compatibilité navigateurs et responsive

L'application est contrôlée sur **Chrome, Firefox et Edge** et sur trois largeurs représentatives : mobile (375 px), tablette (768 px) et ordinateur (1440 px), à l'aide du mode appareil de Chrome DevTools. Les captures de la partie 4.7 documentent ces contrôles.

5.6 Bilan des tests et axes d'amélioration

La recette manuelle et les tests Postman couvrent l'ensemble des parcours critiques ; les tests unitaires couvrent les composants transverses. **Limites** : la collection Postman n'est pas encore rejouée automatiquement et le front-end n'a pas de tests automatisés. **Évolutions prévues** : tests d'intégration de l'API avec Jest et Supertest, exécution de la collection Postman en ligne de commande (Newman), tests de bout en bout (Playwright ou Cypress) et exécution à chaque `push` via GitHub Actions.

Partie 6 — Sécurité : failles majeures et protections

6.1 Cartographie des menaces

La sécurité est abordée au regard du référentiel **OWASP Top 10**. Le tableau synthétise les failles majeures des applications web et leur traitement dans ITConnect ; le détail suit.

Faille	Principe de l'attaque	Protection dans ITConnect	Niveau
Injection SQL	Code SQL glissé dans un champ de saisie pour lire ou modifier la base	Accès aux données uniquement par l'ORM Sequelize (requêtes paramétrées)	Couvert
XSS	Script injecté dans un contenu affiché à d'autres utilisateurs	Échappement automatique de React ; aucun `innerHTML` ni `dangerouslySetInnerHTML`	Couvert (CSP à ajouter)
CSRF	Requête forgée depuis un autre site en profitant de la session de la victime	Jeton envoyé dans l'en-tête `Authorization`, pas dans un cookie ; CORS restreint	Couvert
Authentification faible	Mots de passe devinés ou volés, jeton volé ou périmé	Hachage bcryptjs, jeton signé et expirant, message d'erreur générique	Couvert (limiteur à ajouter)
Contrôle d'accès défaillant	Accès aux ressources d'un autre utilisateur ou à un rôle supérieur	`authentifier` / `autoriser` côté serveur, liste blanche des rôles, champs modifiables restreints	Couvert
Exposition de données	Fuite de secrets ou de données personnelles	Hash retiré des réponses, `.env` non versionné, erreurs 500 sans détail	Couvert
Force brute	Essais massifs de mots de passe	Non limité dans cette version	Prévu
Composants vulnérables	Dépendances npm obsolètes	Versions épinglées par `package-lock.json`, audit `npm audit`	Partiel

6.2 Injection SQL

Principe : si une requête SQL est construite par concaténation de texte saisi, un attaquant peut modifier sa logique (par exemple `' OR '1'='1` dans un champ e-mail pour contourner une authentification). **Protection** : ITConnect n'écrit **aucune requête SQL à la main** dans l'API ; toutes les lectures et écritures passent par Sequelize, qui envoie les valeurs sous forme de **paramètres liés** distincts de la requête. Les modèles contrôlent en outre les types, les longueurs, le format de l'e-mail et la note (1 à 5). Le seul SQL écrit à la main est le script de création de la base.

Requête paramétrée par l'ORM

```
// Les valeurs saisies ne sont jamais concaténées : Sequelize les transmet comme paramètres
const user = await User.findOne({ where: { email } })

// À proscrire (concaténation = injection possible) :
// sequelize.query("SELECT * FROM utilisateurs WHERE email = '" + email + "'")
```

6.3 Cross-Site Scripting (XSS)

Principe : un script malveillant saisi dans un champ (objet d'une demande, texte d'un avis, bio d'un profil) est restitué à d'autres utilisateurs et s'exécute dans leur navigateur (vol de jeton, redirection). **Protection** : React **échappe par défaut** tout contenu inséré dans le JSX ; le code du front-end n'utilise ni `dangerouslySetInnerHTML`, ni `innerHTML`, ni `eval`, ni `document.write` (vérifié par recherche dans le projet). **Limite** : le jeton est conservé dans `localStorage`, accessible à un script en cas de faille XSS ; l'ajout d'une politique **Content-Security-Policy** (module `helmet`) et, à terme, un cookie `httpOnly` réduiraient ce risque.

6.4 Cross-Site Request Forgery (CSRF)

Principe : un site malveillant déclenche, depuis le navigateur d'une victime connectée, une requête vers l'application en profitant des **cookies envoyés automatiquement**. **Protection** : l'authentification repose sur un jeton transmis **explicitement** par le front-end dans l'en-tête `Authorization` et non sur un cookie ; un site tiers ne peut donc pas l'ajouter à une requête forgée. Le **CORS** n'autorise que l'origine du front-end (`FRONTEND_URL`). **Si** le jeton passait un jour dans un cookie `httpOnly`, il faudrait ajouter l'attribut `SameSite` et un jeton anti-CSRF.

6.5 Authentification et gestion des sessions

- **Mots de passe** : hachés avec **bcryptjs** (salage, 10 tours) avant tout enregistrement ; jamais stockés ni renvoyés en clair (`toJSON()` retire le hash des réponses).
- **Politique de mot de passe** : 8 caractères minimum, 1 majuscule, 1 chiffre, contrôlée **côté serveur** en plus du formulaire.
- **Jeton JWT** : signé avec `JWT_SECRET`, durée de validité limitée par `JWT_EXPIRES_IN` (7 jours par défaut) ; en cas d'expiration, réponse 401 « Session expirée. ».
- **Message générique** : « Email ou mot de passe incorrect. » dans tous les cas d'échec, pour ne pas révéler l'existence d'un compte.
- **Relecture en base** : `authentifier` recharge l'utilisateur à chaque requête ; un compte supprimé est refusé immédiatement.
- **Côté client** : sur une réponse 401, l'intercepteur Axios supprime la session et redirige vers la connexion.

6.6 Contrôle d'accès et « mass assignment »

Le **contrôle d'accès** est entièrement appliqué côté serveur (matrice des droits, 2.3). Une revue de code a révélé une faille de type **mass assignment** : la route d'inscription transmettait telle quelle le champ `role` du corps de la requête à la base ; un attaquant pouvait donc créer un compte **administrateur** en envoyant `"role": "admin"`. La correction applique une **liste blanche** : tout rôle absent de la liste devient `particulier`, et les comptes administrateur ne sont créés que par le script de la base.

Correction de la faille d'attribution de rôle (authController.js)

```
// AVANT (vulnérable) : le rôle vient directement de la requête
const user = await User.create({ prenom, nom, email, motDePasse, role })

// APRÈS : liste blanche des rôles
const ROLES_AUTORISES = ['particulier', 'entreprise', 'professionnel']
const roleFinal = ROLES_AUTORISES.includes(role) ? role : 'particulier'
const user = await User.create({ prenom, nom, email, motDePasse, role: roleFinal })
```

- `updateMe` ne lit que `prenom`, `nom`, `email`, `ville` : le **rôle n'est pas modifiable** après l'inscription.
- Le détail d'une demande n'est accessible qu'à son auteur, au professionnel destinataire ou à un administrateur ; seul le destinataire change le statut, selon des transitions valides.
- À la création d'une demande, l'auteur provient du jeton et le statut de sa valeur par défaut ; un professionnel ne peut pas s'adresser une demande à lui-même.

6.7 Exposition de données et configuration

- **Secrets** : `JWT_SECRET`, identifiants de base de données et autres paramètres sont dans un fichier `.env` **exclu de Git** (`.gitignore`) ; en production, ils sont saisis dans les variables d'environnement de l'hébergeur.
- **Erreurs** : le middleware central renvoie un message générique pour toute erreur 500 ; le détail reste dans les journaux serveur.

- **CORS** : origine unique configurée, jamais « * » avec des identifiants.
- **Transport** : HTTPS imposé par l'hébergeur en production (GitHub Pages, Render).
- **Données de démonstration** : comptes fictifs partageant un mot de passe de test, à supprimer ou renouveler avant toute mise en production réelle.

6.8 Limites et améliorations prévues

Point identifié	Risque	Amélioration prévue
Jeton stocké dans `localStorage`	Lisible par un script en cas de XSS	Cookie `httpOnly` + `SameSite` + protection CSRF dédiée
Pas de limitation des tentatives de connexion	Force brute	Module `express-rate-limit` sur `/api/auth`
Pas d'en-têtes de sécurité HTTP	Clickjacking, sniffing de type MIME, XSS	Module `helmet` (Content-Security-Policy, X-Content-Type-Options…)
Validation des entrées limitée aux modèles et à quelques contrôles	Données mal formées	Bibliothèque dédiée (`express-validator` ou `Joi`)
Dépendances non auditées automatiquement	Failles connues	`npm audit` régulier, Dependabot sur GitHub
Comptes de démonstration	Accès non voulu en production	Suppression avant mise en production

Partie 7 — Conformité au RGPD

7.1 Cadre réglementaire

ITConnect traite des **données personnelles** (identité, e-mail, ville, contenu des échanges) de personnes situées en France. Elle est soumise au **Règlement général sur la protection des données** (UE 2016/679) et à la loi « Informatique et Libertés », sous le contrôle de la **CNIL**. L'entreprise ITConnect (fictive) est le **responsable de traitement**. Cette partie décrit les mesures intégrées dès la conception (« privacy by design ») et celles qui restent à mettre en place avant une exploitation réelle.

7.2 Données collectées, finalités et conservation

Donnée	Finalité	Base légale	Conservation proposée
Prénom, nom	Identifier l'utilisateur, afficher les profils	Exécution du contrat (compte)	Durée du compte
Adresse e-mail	Connexion, communications liées au service	Exécution du contrat	Durée du compte
Mot de passe (haché)	Authentification	Exécution du contrat	Durée du compte ; jamais en clair
Rôle, ville	Adapter les droits et la recherche locale	Exécution du contrat	Durée du compte
Profil professionnel (spécialité, bio, tarif, compétences)	Visibilité du professionnel, recherche	Exécution du contrat / consentement (publication du profil)	Durée du compte
Demandes (objet, message)	Mise en relation et suivi des prestations	Exécution du contrat	Durée du compte, puis suppression en cascade
Avis (note, texte)	Confiance entre utilisateurs	Intérêt légitime / consentement	Durée du compte
Journaux techniques de l'hébergeur	Sécurité et exploitation	Intérêt légitime	Durée définie par l'hébergeur

Aucune donnée « sensible » au sens de l'article 9 du RGPD (santé, opinions, origine…) n'est collectée. Aucune donnée n'est vendue ni transmise à des tiers à des fins commerciales.

7.3 Principes appliqués

- **Minimisation** : seules les données nécessaires au service sont demandées à l'inscription (prénom, nom, e-mail, mot de passe, rôle) ; la ville et le profil détaillé sont facultatifs ou renseignés plus tard.
- **Limitation des finalités** : les données ne servent qu'à la mise en relation, à l'authentification et à la confiance (avis).
- **Exactitude** : l'utilisateur peut corriger ses informations à tout moment (`PUT /api/auth/me`).
- **Intégrité et confidentialité** : mots de passe hachés, hash retiré des réponses, contrôle d'accès par rôle, HTTPS, secrets hors du dépôt.
- **Limitation de la conservation** : suppression en cascade avec le compte ; politique de purge des comptes inactifs à définir.

7.4 Droits des personnes et mise en œuvre technique

Droit (articles RGPD)	Mise en œuvre dans ITConnect	État
Accès (art. 15)	`GET /api/auth/me` : consultation des données du compte ; liste de ses demandes (`GET /api/demandes`)	Implémenté
Rectification (art. 16)	`PUT /api/auth/me` (identité, e-mail, ville) ; `PUT /api/professionnels/me` (profil) ; `PUT /api/auth/mot-de-passe`	Implémenté
Effacement (art. 17)	`DELETE /api/auth/me` : suppression du compte et, par ON DELETE CASCADE, du profil, des demandes et des avis	Implémenté
Portabilité (art. 20)	Export JSON des données du compte	Prévu
Opposition, limitation (art. 18 et 21)	Demande par e-mail au responsable de traitement	À formaliser
Information (art. 13)	Page « Politique de confidentialité » et mentions légales	À publier
Consentement (art. 7)	Case à cocher d'acceptation des conditions et de la politique à l'inscription	À ajouter

7.5 Sécurité des données et gestion d'une violation

Les mesures techniques de la partie 6 constituent les mesures de sécurité exigées par l'article 32. En cas de **violation de données** (accès non autorisé, fuite), la procédure prévue est : détection et confinement (révocation de `JWT_SECRET`, ce qui invalide tous les jetons ; rotation des identifiants de base de données), analyse de l'étendue, **notification à la CNIL dans les 72 heures** si un risque pour les personnes existe, information des personnes concernées si le risque est élevé, et consignation dans un registre des violations.

7.6 Cookies et traceurs

L'application **ne dépose aucun cookie** et n'utilise **aucun outil de mesure d'audience ni traceur tiers**. Le jeton d'authentification est conservé dans le `localStorage` du navigateur : il est **strictement nécessaire** au service demandé par l'utilisateur (connexion) et ne requiert donc pas de consentement, mais il doit être mentionné dans la politique de confidentialité.

7.7 Sous-traitants et transferts

Prestataire	Rôle	Point d'attention
GitHub (GitHub Pages)	Hébergement du front-end statique	Aucune donnée personnelle stockée côté front ; adresses IP traitées par l'hébergeur
Render	Hébergement de l'API	Choisir une région européenne ; accord de traitement des données (DPA) ; variables d'environnement pour les secrets
Hébergeur MySQL	Stockage des données	Région européenne, chiffrement du transport (SSL), sauvegardes, DPA

7.8 Actions restantes avant une exploitation réelle

- Rédiger et publier la **politique de confidentialité** et les **mentions légales**, accessibles depuis le pied de page.
- Ajouter la **case de consentement** à l'inscription et l'**export JSON** des données (portabilité).
- Définir et automatiser la **purge** des comptes inactifs (par exemple après 3 ans).
- Tenir un **registre des traitements** (finalités, données, destinataires, durées, mesures de sécurité).
- Conclure les **accords de sous-traitance** avec les hébergeurs et vérifier la localisation des données.

- Remplacer les **données de démonstration** par des comptes réels uniquement après ces étapes.

Partie 8 — Déploiement et hébergement

8.1 Vue d'ensemble des environnements

ITConnect se déploie en **trois environnements** successifs. Le code est versionné sur **GitHub** ; le front-end (fichiers statiques) est publié sur **GitHub Pages**, l'API Node.js sur **Render**, et la base MySQL chez un fournisseur de base de données externe.

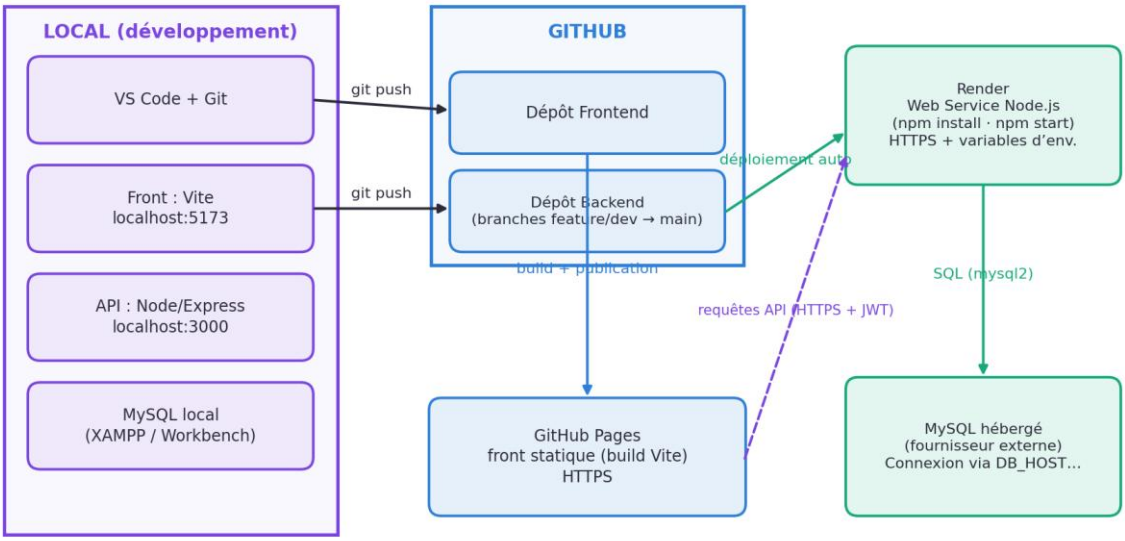


Figure 26 — Environnements et chaîne de déploiement d'ITConnect

Environnement	Front-end	API	Base de données	Usage
Local	Vite — `localhost:5173`	Node/Express — `localhost:3000`	MySQL local (XAMPP / Workbench)	Développement et tests
Production — front	GitHub Pages (HTTPS)	—	—	Interface publique, fichiers statiques
Production — API	—	Render, Web Service Node.js (HTTPS)	MySQL hébergé (région européenne)	API et données

8.2 Environnement local

Prérequis : Node.js 18 ou supérieur, MySQL 8 ou supérieur (serveur local), npm, Git. Procédure :

Mise en route en local

```
# 1. Récupérer le code
git clone <url-du-depot-backend> && cd itconnect-backend

# 2. Installer les dépendances
npm install

# 3. Créer la base et les tables (phpMyAdmin / Workbench, ou en ligne de commande)
mysql -u root -p < itconnect.sql

# 4. Configurer l'environnement : copier .env.example en .env puis renseigner les valeurs
cp .env.example .env

# 5. Lancer l'API (redémarrage automatique avec nodemon)
npm run dev        # -> http://localhost:3000   (GET / confirme que l'API répond)
```

```
# 6. Lancer le front-end, dans le dossier Frontend
npm install && npm run dev      # -> http://localhost:5173
```

Le front-end lit l'adresse de l'API dans `VITE_API_URL` (fichier `.env` du dossier Frontend, valeur `http://localhost:3000`, **sans `/api` final** : les routes l'ajoutent déjà). Les variables Vite ne sont lues qu'au démarrage : relancer `npm run dev` après modification.

8.3 Publication du front-end sur GitHub Pages

GitHub Pages sert des fichiers statiques : on publie donc le **résultat du build** (`npm run build` → dossier `dist`). Trois points demandent de l'attention pour une application React monopage :

- **Chemin de base** : un site de projet est servi sous `https://<utilisateur>.github.io/<depot>/` ; il faut déclarer `base: '/<depot>/'` dans `vite.config.js`.
- **Routage** : un rechargement sur une URL comme `/connexion` renvoie une 404 car ce fichier n'existe pas. Solution : utiliser `HashRouter` (URL de type `/#/connexion`) ou copier `index.html` en `404.html` au moment du build.
- **Adresse de l'API** : `VITE_API_URL` est **injectée au moment du build** ; elle doit pointer vers l'URL publique de l'API sur Render (en HTTPS).

Workflow GitHub Actions de déploiement sur GitHub Pages

```
# .github/workflows/deploy.yml — publication automatique à chaque push sur main
name: Deploy front-end
on:
  push:
    branches: [main]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run build
        env:
          VITE_API_URL: ${{ vars.VITE_API_URL }}
      - uses: actions/upload-pages-artifact@v3
        with: { path: dist }
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: { name: github-pages }
    steps:
      - uses: actions/deploy-pages@v4
```

Dans les réglages du dépôt (Settings → Pages), la source de publication est réglée sur **GitHub Actions**. L'URL publique est : [À COMPLÉTER : URL GitHub Pages du front-end].

8.4 Déploiement de l'API sur Render

Render déploie l'API directement depuis le dépôt GitHub et redéploie automatiquement à chaque `push` sur la branche suivie. Création d'un **Web Service** :

Paramètre Render	Valeur
Source	Dépôt GitHub du back-end, branche `main`

Paramètre Render	Valeur
Runtime	Node
Build Command	`npm install`
Start Command	`npm start` (soit `node server.js`)
Instance	Free pour la démonstration (voir limites ci-dessous)
Variables d'environnement	Voir tableau 8.5 ; PORT est fourni par Render, `server.js` le lit déjà

Base de données. Render propose des bases managées PostgreSQL et Key Value, mais **pas de MySQL managé** : la base MySQL du projet est donc hébergée chez un **fournisseur externe** (par exemple Aiven ou Railway, offres d'essai ou gratuites), puis alimentée en important `itconnect.sql`. Les identifiants fournis (hôte, port, utilisateur, mot de passe, nom de base) sont saisis dans les variables `DB_*` de Render. Les connexions distantes exigent souvent **SSL** ; l'option se règle dans `config/database.js` :

Évolution de config/database.js : SSL activable par variable d'environnement

```
const sequelize = new Sequelize(process.env.DB_NAME, process.env.DB_USER, process.env.DB_PASSWORD, {
  host: process.env.DB_HOST, port: process.env.DB_PORT || 3306, dialect: 'mysql',
  dialectOptions: process.env.DB_SSL === 'true' ? { ssl: { rejectUnauthorized: true } } : {},
  define: { charset: 'utf8mb4', collate: 'utf8mb4_unicode_ci', timestamps: true },
})
```

> **Limites de l'offre gratuite de Render** : un service gratuit est **mis en veille après environ 15 minutes sans trafic** et la première requête suivante subit un démarrage à froid pouvant durer près d'une minute ; le système de fichiers est éphémère et les heures gratuites sont plafonnées chaque mois. Cette offre convient à une démonstration, pas à une exploitation réelle (offre payante ou autre hébergeur à prévoir).

8.5 Variables d'environnement par environnement

Variable	Local	Production (Render)	Rôle
PORT	3000	Fourni par Render	Port d'écoute de l'API
DB_HOST / DB_PORT	localhost / 3306	Hôte et port du MySQL hébergé	Connexion à la base
DB_NAME / DB_USER / DB_PASSWORD	itconnect / root / mot de passe local	Identifiants du fournisseur	Authentification MySQL
DB_SSL	absent	true	Chiffrement de la connexion à la base
JWT_SECRET	chaîne de test	Chaîne longue et aléatoire, unique	Signature des jetons
JWT_EXPIRES_IN	7d	7d (ou plus court)	Durée de validité des jetons
FRONTEND_URL	http://localhost:5173	`https://<utilisateur>.github.io` (origine, sans chemin)	Origine autorisée par CORS
VITE_API_URL (front)	http://localhost:3000	URL publique de l'API Render	Adresse de l'API, injectée au build

8.6 Versionnement et flux de branches

Le code est versionné avec **Git** et hébergé sur GitHub. Le travail se fait sur la branche **`feature/dev`** ; l'intégration dans **`main`** (branche déployée) passe par une *pull request*. Chaque étape fonctionnelle fait l'objet d'un commit au message explicite, ce qui permet de retracer l'évolution du projet et de revenir à une version stable.

Flux de travail Git

```
git add .
git commit -m "Correction de la liste blanche des rôles (authController)"
git push -u origin feature/dev      # puis pull request vers main sur GitHub

# Mise à jour de main en local après fusion
git checkout main && git pull origin main
```

Un incident illustre l'intérêt de ce flux : un `git push origin main` depuis une branche locale en retard a été **rejeté (non-fast-forward)** par GitHub. La solution n'a pas été de forcer le push, qui aurait écrasé l'historique distant, mais de pousser `feature/dev`, de fusionner par pull request puis de mettre `main` à jour avec `git pull`.

8.7 Liste de contrôle avant mise en production

Point de contrôle	Pourquoi	Fait
Secrets (`JWT_SECRET`, mots de passe) uniquement dans les variables d'environnement	Aucun secret dans le dépôt	☐
`.env` listé dans `.gitignore`, historique Git vérifié	Éviter la fuite d'identifiants	☐
Comptes de démonstration supprimés ou mots de passe renouvelés	Pas d'accès administrateur connu	☐
CORS limité à l'origine du front-end publié	Empêcher les appels depuis d'autres sites	☐
HTTPS sur le front-end et l'API ; SSL vers la base	Chiffrer le transport	☐
`helmet` et `express-rate-limit` activés	En-têtes de sécurité, force brute	☐
Politique de confidentialité, mentions légales, consentement	Conformité RGPD	☐
Sauvegarde régulière de la base (`mysqldump`) testée en restauration	Reprise après incident	☐
`npm audit` sans vulnérabilité critique	Dépendances saines	☐

8.8 Maintenance

La maintenance courante comprend la mise à jour des dépendances (`npm outdated`, `npm audit`), la sauvegarde périodique de la base, la surveillance des journaux de l'hébergeur et le renouvellement des secrets en cas de doute. Les redéploiements sont automatiques à chaque fusion dans `main`.

Partie 9 — Gestion de projet

9.1 Méthodologie

Le projet a été mené en solo, avec une organisation **inspirée des méthodes agiles** : backlog de user stories, découpage en tâches courtes, priorisation (parcours principal d'abord) et points d'étape réguliers avec les formateurs. Le suivi des tâches a été assuré sur support papier, complété par l'historique des commits Git.

9.2 Planning

Phase	Contenu	Durée indicative
Cadrage	Besoin, user stories, périmètre fonctionnel	1 à 2 semaines
Conception	Maquettes, MCD, architecture, charte graphique	2 semaines
Back-end	API REST, modèles Sequelize, authentification	3 à 4 semaines
Front-end	Pages React, intégration CSS, connexion à l'API	4 semaines
Intégration et tests	Connexion front/back, recette, tests Postman	2 semaines
Finalisation	Corrections, sécurité, documentation, déploiement	2 semaines

9.3 Outils de suivi

- **Git / GitHub** : versionnement, branches, pull requests.
- **Figma** : maquettes et prototype.
- **MySQL Workbench** : contrôle de la base et des jointures.
- **Postman** : tests de l'API.
- **README** : installation, variables d'environnement, structure, routes et rôles.

9.4 Veille technologique

Thématique	Sources	Application au projet
Sécurité des applications web	OWASP Top 10, documentation JWT, CNIL	Liste blanche des rôles, hachage, CORS, plan d'amélioration (helmet, limitation de débit)
Écosystème React et Vite	Documentation officielle React et Vite	Choix de Vite, composants, services Axios
ORM Sequelize	Documentation officielle Sequelize	Associations, modes de synchronisation, requêtes paramétrées
Accessibilité	Référentiel WCAG 2.1	Contrastes, navigation au clavier, attributs ARIA
Déploiement	Documentation GitHub Pages, GitHub Actions, Render	Procédure de la partie 8, limites de l'offre gratuite

9.5 Budget indicatif de mise en production

Le développement n'a mobilisé **aucun budget externe**. Une estimation mensuelle indicative pour une exploitation réelle : hébergement de l'API et de la base (20 à 40 €), nom de domaine et sauvegardes (5 à 10 €), outils de communication (50 à 100 €), divers (30 €).

Partie 10 — Difficultés rencontrées et solutions apportées

Les difficultés ci-dessous ont été rencontrées au fil du projet ; chacune a conduit à une solution précise et à un enseignement réutilisable.

Difficulté	Cause identifiée	Solution apportée	Enseignement
Conflits de ports en développement	Front (Vite) et API sur des ports par défaut proches, processus déjà en écoute	Ports configurés explicitement (5173 / 3000) et variables d'environnement dédiées	Ne jamais dépendre d'un port implicite
Synchronisation Sequelize incohérente	Modes `alter` / `force` modifiant ou supprimant des tables	`sync({ force: false })` en usage courant ; script `itconnect.sql` comme référence du schéma	Distinguer environnement de développement et données à préserver
Mots de passe de test refusés à la connexion	Hash bcrypt de test incompatible avec la vérification de l'API	Un seul point de hachage (modèle `User`), même bibliothèque (`bcryptjs`) et 10 tours de salage	Centraliser les opérations sensibles
Requêtes MySQL bloquées dans Workbench	Mode « safe update » refusant UPDATE/DELETE sans clé primaire	Requêtes de test écrites avec la clé primaire explicite	Lire les messages d'erreur de l'outil avant de le contourner
Appels réseau dispersés dans les composants	Configuration Axios répétée dans chaque composant	Services `authService` et `proService` avec intercepteurs ; regroupement en un module unique prévu	Factoriser tôt les accès réseau
URL d'API en double (`/api/api/…`)	Adresse de base par défaut terminée par `/api` alors que les routes l'incluent déjà	Adresse de base ramenée à `http://localhost:3000` ; `VITE_API_URL` documentée	Vérifier les requêtes réelles dans l'onglet Réseau
Faille d'attribution du rôle `admin`	Champ `role` du corps de la requête transmis tel quel à la base (mass assignment)	Liste blanche des rôles côté serveur ; mot de passe contrôlé côté serveur ; tests TF3 et TF4	Ne jamais faire confiance aux données du client
Push Git rejeté (non-fast-forward)	Branche locale `main` en retard sur celle de GitHub	Poussée de `feature/dev`, pull request, puis `git pull` sur `main` ; pas de push forcé	Travailler sur des branches de fonctionnalité
Accents et emojis mal affichés	Encodage non uniforme entre base, API et navigateur	`utf8mb4` en base, en-tête `charset=utf-8` côté API	Fixer l'encodage de bout en bout
Appels bloqués entre front et API	Politique CORS trop ou pas assez permissive	Origine unique configurée par `FRONTEND_URL`, variable différente selon l'environnement	Une origine par environnement, jamais « * »
Icône de recherche mal placée en mobile	Icône hors du champ lors de l'empilement responsive (visible sur les captures)	Correction à appliquer : conteneur en `position: relative`, icône en `position: absolute`	Tester chaque composant sur les trois largeurs

10.1 Analyse transversale

Trois enseignements se dégagent. **Premièrement**, les erreurs les plus coûteuses viennent de **configurations implicites** (ports, URL de base, encodage, origines CORS) : les rendre explicites et les documenter dans le README fait gagner du temps à chaque installation. **Deuxièmement**, la faille d'attribution du rôle a été détectée par **relecture de code** et non par un test : elle montre qu'un contrôle d'accès doit être pensé côté serveur dès la conception et vérifié par des tests dédiés (TF3, TF12). **Troisièmement**, travailler seul impose une discipline de **versionnement** et de **documentation** qui compense l'absence de relecture par un pair.

Partie 11 — Bilan, prise de recul et conclusion générale

11.1 Bilan technique : objectifs atteints

Objectif	État	Commentaire
Recherche de professionnels avec filtres	Atteint	Spécialité, ville, disponibilité, note ; grille ou liste
Comptes, authentification et rôles	Atteint	JWT, bcryptjs, contrôle d'accès côté serveur, liste blanche des rôles
Demandes et avis	Atteint	Création, suivi, un avis par couple client / professionnel
Interface responsive et accessible	Atteint avec réserves	Trois formats ; quelques ajustements (icônes de recherche, contrastes du bleu)
Sécurité	Atteint avec réserves	Failles majeures traitées ; `helmet`, limitation de débit et cookie `httpOnly` à ajouter
Conformité RGPD	Partiel	Droits d'accès, rectification et effacement implémentés ; politique de confidentialité, consentement et portabilité à ajouter
Déploiement	Procédure définie	GitHub Pages et Render documentés ; offre gratuite limitée à la démonstration
Tests automatisés	Partiel	Postman et recette manuelle ; Jest sur les middlewares ; couverture à étendre

11.2 Compétences du titre mobilisées

Bloc de compétences	Mise en œuvre dans ITConnect	Parties
CCP1 — Développer la partie front-end d'une application web ou web mobile	Maquettage (zoning, wireframes, Figma), interfaces statiques et responsive, interface dynamique connectée à l'API, accessibilité	2.7, 4.4 à 4.8
CCP2 — Développer la partie back-end d'une application web ou web mobile	Base de données relationnelle, composants d'accès aux données, API REST sécurisée, documentation du déploiement	3, 4.3, 6, 8
Transversales	Analyse du besoin, tests, sécurité, RGPD, gestion de projet en autonomie, veille	1, 2, 5, 7, 9

11.3 Prise de recul

Ce qui a bien fonctionné. La séparation nette entre front-end et back-end, formalisée par un contrat d'API simple, a permis de développer et de tester chaque partie indépendamment. L'usage d'un ORM a supprimé toute requête SQL écrite à la main dans l'API, réduisant mécaniquement le risque d'injection. La conception des maquettes avant l'intégration a limité les retours en arrière sur l'interface.

Ce que je ferais autrement. J'écrirais les **tests dès les premières routes** plutôt qu'en fin de projet : la faille d'attribution du rôle aurait été détectée par un test avant d'être repérée en relecture. Je mettrais en place **plus tôt** la validation centralisée des entrées, un module unique d'accès à l'API côté front-end et une intégration continue exécutant tests et audit des dépendances. J'aurais aussi anticipé la **conformité RGPD** (consentement, politique de confidentialité) dès la conception de l'inscription plutôt qu'en fin de projet.

Limites. Le projet est réalisé seul et sans retours d'utilisateurs réels : les parcours ont été validés par moi-même et par des données de démonstration. L'espace d'administration, le paiement et la prise de rendez-vous ne sont pas développés, et l'hébergement gratuit ne garantit ni la disponibilité ni les performances d'une exploitation réelle.

11.4 Perspectives d'évolution

- Espace d'administration : modération des comptes, des avis et des demandes, statistiques globales.
- Messagerie intégrée à une demande et notifications (nouvelle demande, changement de statut).
- Paiement en ligne sécurisé et prise de rendez-vous.
- Application mobile dédiée en complément de la version web responsive.
- Intégration continue (tests Jest, collection Postman via Newman, audit des dépendances) et tests de bout en bout.
- Mise en conformité RGPD complète et durcissement de la sécurité (cookie `httpOnly`, `helmet`, limitation de débit).

11.5 Conclusion générale

ITConnect m'a permis de conduire, **de bout en bout et en autonomie**, la réalisation d'une application web : analyse du besoin, conception (maquettes, MCD, diagramme de classes, architecture), développement d'une API REST sécurisée et d'une interface React responsive, tests, prise en compte de la sécurité et du RGPD, puis définition d'une chaîne de déploiement. Le résultat est une plateforme fonctionnelle dont le parcours principal — de l'inscription au suivi d'une demande — est opérationnel et documenté.

Ce projet m'a surtout appris que la qualité d'une application se joue autant dans **ce que l'on ne voit pas** — contrôle d'accès côté serveur, validation des données, gestion des erreurs, configuration explicite, traçabilité du code — que dans l'interface. La faille d'attribution du rôle détectée puis corrigée en est l'illustration la plus concrète : elle m'a conduit à adopter une démarche de relecture critique et à placer les tests au centre de ma méthode de travail.

Enfin, la démarche de prise de recul menée dans ce dossier — limites assumées, améliorations priorisées, perspectives réalistes — correspond à la posture attendue d'un développeur web et web mobile : livrer une solution utile, expliquer ses choix, reconnaître ses faiblesses et savoir comment les corriger.

Annexes

Annexe A — Index des routes de l'API

Méthode	Route	Description	Accès
POST	/api/auth/inscription	Création d'un compte	Public
POST	/api/auth/connexion	Authentification et génération du jeton	Public
GET	/api/auth/me	Consultation du compte connecté	Authentifié
PUT	/api/auth/me	Modification du compte connecté	Authentifié
PUT	/api/auth/mot-de-passe	Changement du mot de passe	Authentifié
DELETE	/api/auth/me	Suppression du compte (droit à l'effacement)	Authentifié
GET	/api/professionnels	Liste et recherche des professionnels	Public
GET	/api/professionnels/:id	Détail d'un profil professionnel	Public
PUT	/api/professionnels/me	Mise à jour du profil du professionnel connecté	Professionnel
GET	/api/professionnels/:id/avis	Liste des avis d'un professionnel	Public
POST	/api/professionnels/:id/avis	Dépôt d'un avis	Particulier, Entreprise
GET	/api/demandes	Liste des demandes de l'utilisateur connecté	Authentifié
POST	/api/demandes	Création d'une demande	Particulier, Entreprise
GET	/api/demandes/:id	Détail d'une demande	Auteur, professionnel concerné, administrateur
PUT	/api/demandes/:id/statut	Mise à jour du statut	Professionnel destinataire
GET	/api/entreprises (/:id)	Liste et détail des entreprises	Public

Annexe B — Variables d'environnement (extrait du README)

Fichier .env.example du back-end (valeurs d'exemple)

```
# ── Serveur ──
PORT=3000
# ── Base de données MySQL ──
DB_HOST=localhost
DB_PORT=3306
DB_NAME=itconnect
DB_USER=root
DB_PASSWORD=your_password_here
# ── JWT ──
JWT_SECRET=change_this_in_production
JWT_EXPIRES_IN=7d
# ── CORS ──
FRONTEND_URL=http://localhost:5173
```

Annexe C — Glossaire

Terme	Définition
API REST	Interface de programmation exposée par des routes HTTP, échangeant des données en JSON
JWT	JSON Web Token : jeton signé permettant d'authentifier un utilisateur sans session serveur
ORM	Mapping objet-relationnel : correspondance entre objets applicatifs et tables (ici Sequelize)
CORS	Mécanisme contrôlant les origines autorisées à appeler une API
CSRF	Requête forgée depuis un site tiers en exploitant la session d'un utilisateur

Terme	Définition
XSS	Injection de script dans une page affichée à d'autres utilisateurs
Mass assignment	Attribution de champs sensibles à partir de données non filtrées de la requête
Middleware	Fonction exécutée entre la réception d'une requête et son traitement final
MCD	Modèle conceptuel de données (Merise)
RGPD	Règlement général sur la protection des données
SPA	Application monopage : une seule page HTML dont le contenu est géré en JavaScript
CSS Modules	Styles CSS limités à un composant, évitant les conflits de noms